# Vaeltrix V0 Implementation Guide: Contracts and Backend

Oct 17, 2026 · @david · Derived from Vaeltrix Technical Specification v0.1 (Oct 11, 2026)

## 0. How to use this document

This guide is the build contract for Vaeltrix V0: every contract function, storage key, service, table, test and task needed to reach a working testnet loop, written so a human or an AI agent can execute it one section at a time.

The technical specification says what Vaeltrix is. This guide says exactly how V0 gets built. Where the two disagree on implementation detail, this guide wins and Section 2 records why. Where they disagree on product intent, the spec wins and this guide must be corrected.

### 0.1 Reading order

1. Humans: Sections 1 and 2 first (scope and the decisions that change the spec), then the section for the component you own.
2. AI agents: Section 17 (operating rules), then Sections 2, 3 and 4, then only the section named in the issue you are working on. Never start coding a contract before reading Section 4: every contract imports those types.
3. Project leads turning this into GitHub issues: Section 16. Every `VX-xxx` block there is a complete issue body with labels, dependencies and acceptance criteria.

### 0.2 Conventions used everywhere

| Topic | Rule |
|---|---|
| Normative words | MUST and MUST NOT are requirements. SHOULD is the default unless an issue says otherwise. MAY is optional. |
| Section references | §6.3 means Section 6.3 of this guide. "Spec §7.3" means the technical specification. |

| Topic | Rule |
|---|---|
| Issue IDs | `VX-` plus three digits. The hundreds digit is the epic: VX-1xx is foundations, VX-2xx registry, and so on (§16). |
| Amounts | `i128` in the asset's smallest unit. USDC on Stellar has 7 decimals, so 1 USDC = `10_000_000`. Never use floats for money in any language. |
| Payment timestamps | `u64` Unix milliseconds: `t0_ms`, `t1_ms`, `target_ms`, `grace_ms`. |
| Windows and protocol clocks | `u64` Unix seconds: claim, challenge, attestation, exit cooldown, timelock. Ledger time is `env.ledger().timestamp()` in seconds. |
| Converting | On chain, "now in ms" is `env.ledger().timestamp() * 1000`. Off chain, use the ledger close time of the transaction, never the server clock, for anything that becomes evidence. |
| Hashes | SHA-256, 32 bytes, lowercase hex off chain, `BytesN<32>` on chain. |
| Off-chain JSON hashing | RFC 8785 canonical JSON (JCS), then SHA-256. Replaces the CBOR mentioned in spec §6.6 (see §2, D3). |
| Naming | Rust: snake_case. TypeScript: camelCase in code, snake_case in JSON on the wire and in Postgres. Contract event topics are short symbols (max 9 chars for `symbol_short!`). |
| Networks | `testnet` is the only target this weekend. Every script takes `--network` and reads contract IDs from `deployments/<network>.json`. |

## 0.3 What "done" means for any task

- Code compiles with zero warnings (`cargo clippy -- -D warnings`, `tsc --noEmit`).
- Tests named in the issue exist and pass in CI.

- Public interfaces match this guide exactly. A change to an interface MUST update this guide in the same pull request.
- No TODO without a linked issue number.

# 1. Weekend scope and definition of done

By Sunday night (Oct 18, 2026) the target is one scripted run on Stellar testnet where a provider stakes, a payment completes late, a claim is filed by the observer, and the client is paid from stake with no human step.

Everything in this guide is in scope for V0. Not all of it is in scope for this weekend. The table below is the cut line. Issues in §16 carry a `weekend` label when they sit above it.

## 1.1 The cut line

| Area | This weekend (must) | This weekend (stretch) | After the weekend |
|---|---|---|---|
| Shared crates | `vaeltrix-types`, `vaeltrix-verdict` with test vectors | TS port of verdict passing the same vectors | Verdict compiled to wasm for the observer |
| `vaeltrix_registry` | Providers, templates, observers, admin, timelock as a parameter | `verify_domain` driven by a verifier script | Governance council, multisig |
| `vaeltrix_bond` | Commit, add stake, observer-attested claims, challenge, attest, resolve, exit | Provider-signed receipt claims (§2, D2 path B) | Upgrade timelock queue |
| `vaeltrix_scoreboard` | `record_claim_outcome`, `anchor_batch`, views | Lifetime stats roll-up | Multi-observer root matching |
| Observer | Ledger lookup for t0, signed event ingest, breach detector, auto-claim, keeper that calls `resolve` | SEP-31 poller, batch publisher | Attester on challenges against real anchors |

| Area | This weekend (must) | This weekend (stretch) | After the weekend |
|---|---|---|---|
| Indexer and API | Soroban event indexer into Postgres, read endpoints for providers, commitments, claims | Rankings endpoint | Explorer UI |
| SDK | Generated bindings plus thin wrapper for the demo | Full surface in spec §18.2 | npm publish |
| Anchors | Mock SEP-31 anchor with configurable payout delay | Second mock anchor (two-provider ranking) | Real pilot anchors |
| Demo | `pnpm demo:breach` script end to end on testnet | `pnpm demo:challenge` | Recorded video for SCF |

## 1.2 Definition of done for the weekend

- [ ] Three contracts deployed to testnet, IDs committed in `deployments/testnet.json`.
- [ ] All six bond invariants (§6.9) covered by unit tests, and a property test runs at least 1,000 random sequences in CI.
- [ ] Demo template 901 (§6.10, two-minute target, short windows) proposed and activated on testnet.
- [ ] One provider registered, domain-verified (manually is fine) and committed with test USDC stake.
- [ ] `pnpm demo:breach` runs unattended: tags a payment, pays on chain, mock anchor completes late, observer files the claim, keeper resolves after the challenge window, client balance rises by the exact penalty.
- [ ] The indexer shows the claim lifecycle and the API returns it at `GET /v1/claims/:id`.
- [ ] README in the repo root explains how to rerun the demo from a clean checkout in under 15 minutes.

## 1.3 Suggested split across a four-person team

| Lane | Owns | First deliverable (Saturday) | Second deliverable (Sunday) |
|---|---|---|---|
| A: Contracts core | Shared crates, `vaeltrix_bond` | Types, verdict crate, commitments and stake | Claims, challenge, resolve, invariant tests |
| B: Contracts edge | `vaeltrix_registry`, `vaeltrix_scoreboard`, deploy scripts | Registry with tests, scoreboard skeleton | Testnet deploy, template activation, bindings |
| C: Observer | Observer service, Postgres schema | Ledger t0 lookup, signed event ingest | Breach detector, auto-claim, keeper |
| D: Glue | Mock anchor, SDK wrapper, indexer, API, demo script | Mock anchor and signing library | Indexer, API, demo script |

Lanes A and B unblock C and D through §4 (types) and §4.4 (signed message layout). Agree those two on Saturday morning before anything else and freeze them.

## 1.4 Order of work

1. Freeze §4 types and §4.4 byte layouts (one hour, all lanes together).
2. Scaffold the repo (§3) and CI.
3. Registry and verdict crate in parallel with the mock anchor and signing library.
4. Bond commitments and stake, then claims.
5. Scoreboard.
6. Deploy to testnet, generate bindings.
7. Observer against the deployed contracts.
8. Indexer and API.
9. Demo script, then README.

If time runs short, cut from the bottom of the stretch column first, then the indexer and API. The contract loop plus the observer plus the demo script is the minimum that proves the protocol.

# 2. Spec gaps and V0 implementation decisions

Sixteen places in spec v0.1 cannot be built exactly as written on Soroban or leave a choice open; each is resolved below and every later section assumes these resolutions.

The biggest one is D2: a Soroban contract cannot read past ledger transactions or any off-chain SEP-31 state, so spec §12.1 checks 2, 5 and 6 cannot be proven on chain. V0 moves that proof into an observer attestation at filing time, which matches the V0 trust model in spec §13.2 (one reference observer, testnet only).

## D1. Time units

- **Gap:** the spec mixes milliseconds (t0, t1, targets) with ledger time, which Soroban exposes in seconds.
- **Decision:** payment timestamps and SLA targets stay in milliseconds as in spec §6.2. All windows (`claim_window_secs`, challenge, attestation, cooldown, timelock) are seconds. The contract computes `now_ms = ledger.timestamp() * 1000` only when comparing against payment timestamps.
- **Consequence:** t0 from a ledger close is always a multiple of 1,000 ms. That is expected.

## D2. What the contract can and cannot verify at filing

- **Gap:** spec §12.1 asks `file_claim` to check corridor scope, that the evidence bundle includes the t0 ledger transaction, and that the breach holds on submitted timestamps. A contract can recompute a hash but cannot fetch a past transaction or a SEP-31 status.
- **Decision:** two filing paths.
  - **Path A, observer-attested (weekend must):** `file_claim` is called by, or carries the authorisation of, a registered active observer. The observer has already verified t0 against the ledger, t1 against the provider's signature, scope and carve-outs off chain. The contract re-checks everything it can: commitment coverage window, amount bounds, asset, claim window, duplicate, the breach condition on the submitted numbers, stake availability.
  - **Path B, provider-signed receipt (stretch):** anyone may file with a `CompletionReceipt` signed by one of the provider's registered signer keys that contains both `t0_ms` and `t1_ms`. The contract verifies the ed25519 signature with `env.crypto().ed25519_verify` and computes the breach from the provider's own signed numbers. This covers late completion only; non-completion always uses Path A.

- **Consequence:** in V0 the reference observer is trusted for t0 and scope. This is already the spec's stated V0 assumption.

### D3. Encodings for anything signed or hashed

- **Gap:** the spec names canonical CBOR for evidence and JSON for events and attestations, and signs JSON. Contracts cannot parse JSON or CBOR cheaply.
- **Decision:** two encodings with two jobs.
  - Messages verified on chain (Path B receipts) use a fixed-width big-endian byte layout defined in §4.4, built identically in Rust and TypeScript.
  - Everything only hashed or verified off chain (evidence bundles, signed events, attestation documents) uses RFC 8785 canonical JSON, SHA-256. The contract stores these hashes as opaque `BytesN<32>`.

### D4. Identifier derivation needs an exact byte recipe

- **Decision:**
  - `provider_id = sha256("VAELTRIX:PROVIDER:v1" || home_domain_utf8_lowercase || admin_scaddress_xdr)`.
  - `payment_id = sha256("VAELTRIX:PAYMENT:v1" || sep31_tx_id_utf8 || provider_id)`.
  - The contract recomputes both. `file_claim` takes `sep31_tx_id` so `payment_id` can be checked on chain, which closes a cheap spoofing path.
  - Domain-separation prefixes are ASCII bytes, no length prefix, no separator.

### D5. Tagging is off chain, so solvency is enforced at claim time

- **Gap:** spec §6.3 says a provider may only tag a payment if available stake covers its max penalty, but tagging never touches the contract.
- **Decision:** the observer refuses to accept a tag (HTTP 409) unless `available_stake >= max_penalty(template)` at that moment, and the SDK checks the same view before returning a `vaeltrix_payment_id`. On chain, `file_claim` reserves `min(available, penalty)` and records the shortfall in `Claim.shortfall`. Shortfall counts against the provider on the scoreboard.
- **Why not reject the claim:** rejecting would punish the client for the provider's under-collateralisation.

### D6. "Commitment was Active at t0" needs history

- **Decision:** `Commitment` stores `effective_from`, `paused_at: Option<u64>`, `resumed_at: Option<u64>` and `exit_requested_at: Option<u64>` (all seconds). A payment is covered when `t0_s >= effective_from`, t0 is not inside the most recent pause window, and t0 is before `exit_requested_at`. Only the latest pause window is kept in V0; a second pause overwrites the first. Recorded as a known limitation.

### D7. Claim bond asset and size

- **Decision:** the claim bond is paid in the commitment's stake asset (USDC). Size = `max(min_claim_bond, penalty * claim_bond_bps / 10_000)`, defaults `min_claim_bond = 10_000_000` (1 USDC) and `claim_bond_bps = 1_000` (10%). When the observer files on a client's behalf (delegated), the observer posts the bond and receives the refund.

### D8. Stake asset equals payment asset in V0

- **Decision:** `commit` MUST reject unless `stake_asset == template.scope.asset`. Penalties are computed on the payment amount and paid in the same asset. Spec §15.6 already says USDC only.

### D9. Observer attestation signature is redundant on chain

- **Gap:** spec §10.2 passes both an observer `Address` and a `sig: BytesN<64>`.
- **Decision:** `attest` calls `observer.require_auth()`, which already proves the observer signed. The 64-byte sig is replaced by `attestation_hash: BytesN<32>`, the JCS hash of the full attestation document the observer publishes to its log (spec §13.3). Same for `anchor_batch`.

### D10. Claim status set

- **Decision:** `Filed`, `Challenged`, `Paid`, `Rejected`, `Expired`. The spec's `Upheld` state is transient (upheld and paid happen in the same `resolve` call), so it is an outcome in the `claim_res` event, not a stored status.

### D11. Who calls `resolve`

- **Decision:** permissionless, as in spec. The observer runs a keeper loop that calls `resolve` on every claim whose deadline has passed. Anyone else may too.

## D12. Governance in V0

- **Decision:** one `admin` address per deployment (use a multisig Stellar account on mainnet, a single key on testnet). Timelocked actions (template activation, observer changes, parameter changes, upgrades) go through `propose` then `execute` after `timelock_secs`. `timelock_secs` is set at construction: 7 days on mainnet, 60 seconds on testnet for the weekend.

## D13. Rejected-claim bond split

- **Decision:** `rejected_bond_provider_bps` (default 5,000) of the bond goes to the provider admin, the rest is split equally between observers that voted to reject, remainder dust to the provider.

## D14. Periods for the scoreboard

- **Decision:** a period is a UTC day. `period = period_start_secs / 86_400`. `anchor_batch` requires `period_end - period_start == 86_400` and `period_start % 86_400 == 0`. One batch per (commitment, period); a second is rejected in V0.

## D15. Contract construction and the circular reference

- **Gap:** bond calls scoreboard and scoreboard only accepts calls from bond.
- **Decision:** deploy order registry, then scoreboard (constructor takes registry and admin), then bond (constructor takes registry and scoreboard), then admin calls `scoreboard.set_bond(bond)` exactly once. Constructors use Soroban `__constructor`.

## D16. Short windows for the testnet demo

- **Gap:** spec templates have 7-day claim windows and 48-hour challenge windows, which make a weekend demo impossible.
- **Decision:** challenge and attestation windows become per-deployment parameters (`challenge_window_secs`, `attestation_window_secs`) instead of constants, and a demo template 901 (§6.10) uses a two-minute target, 3-minute challenge window and 10-minute claim window. Production templates 1 to 3 keep the spec values.

# 3. Repository layout, toolchain and CI

One monorepo holds the Rust contracts (Cargo workspace) and the TypeScript services (pnpm workspace), so a type change and its consumers land in one pull request.

## 3.1 Directory tree

```
vaeltrix/
├── Cargo.toml                    # Rust workspace root
├── rust-toolchain.toml
├── package.json                  # pnpm workspace root, scripts below
├── pnpm-workspace.yaml
├── docker-compose.yml            # postgres, observer, indexer, api, mock-
anchor
├── .env.example
├── crates/
│   ├── vaeltrix-types/           # #[contracttype] structs, enums, errors,
client traits
│   └── vaeltrix-verdict/         # pure no_std breach + penalty + carve-out
logic
├── contracts/
│   ├── registry/                 # vaeltrix_registry
│   ├── bond/                     # vaeltrix_bond
│   └── scoreboard/               # vaeltrix_scoreboard
├── test-vectors/
│   ├── verdict.json              # shared by Rust and TS verdict tests
│   ├── ids.json                  # provider_id and payment_id derivations
│   └── receipts.json             # §4.4 byte layouts + ed25519 signatures
├── packages/
│   ├── core/                     # @vaeltrix/core: JCS, ids, byte layouts,
signing, verdict port
│   ├── bindings/                 # generated by stellar CLI, one folder per
contract
│   │   ├── registry/
│   │   ├── bond/
│   │   └── scoreboard/
│   └── sdk/                      # @vaeltrix/sdk
├── services/
│   ├── observer/                 # vaeltrix-observer (Node, TypeScript)
│   ├── indexer/                  # vaeltrix-indexer
│   ├── api/                      # vaeltrix-api (Fastify)
│   └── mock-anchor/              # SEP-31-shaped receiving anchor for demos
├── db/
```

```
|   └── migrations/                # SQL migrations, numbered
├── scripts/
|   ├── deploy.ts                  # build, deploy, construct, write
deployments/<net>.json
|   ├── seed.ts                    # test USDC, templates, observer, provider,
commitment
|   ├── bindings.sh                # regenerate packages/bindings/*
|   └── demo/
|       ├── breach.ts
|       └── challenge.ts
├── deployments/
|   └── testnet.json
└── docs/
    └── implementation-guide.md   # export of this document
```

## 3.2 Toolchain

Pin versions in the repo; do not rely on whatever is installed globally.

| Tool | Version | Notes |
|---|---|---|
| Rust | stable, pinned in `rust-toolchain.toml` | Target `wasm32v1-none` (the current Soroban target). |
| `soroban-sdk` | `27.x` | Latest major as of Oct 2026. Before pinning, confirm testnet's protocol with `stellar network info --network testnet` and use the SDK major that matches it. |
| `stellar` CLI | `27.x` | `stellar contract build` now optimises wasm by default, no extra flag needed. |
| `proptest` | `1.x` | Dev dependency for bond property tests. |
| Node.js | 22 LTS | All services. |
| pnpm | 9.x | Workspaces. |
| TypeScript | 5.x, `strict: true` | |

| Tool | Version | Notes |
|---|---|---|
| `@stellar/stellar-sdk` | latest major | RPC client, XDR, keypairs, contract client used by generated bindings. |
| Postgres | 16 | Via Docker Compose locally. |
| Fastify | 5.x | API and observer HTTP. |
| Kysely | latest | Typed SQL. Migrations are plain SQL files. |
| Vitest | latest | TypeScript tests. |

## 3.3 Root Cargo workspace

```
[workspace]
resolver = "2"
members = ["crates/*", "contracts/*"]

[workspace.dependencies]
soroban-sdk = "27"
vaeltrix-types = { path = "crates/vaeltrix-types" }
vaeltrix-verdict = { path = "crates/vaeltrix-verdict" }

[profile.release]
opt-level = "z"
overflow-checks = true
debug = 0
strip = "symbols"
debug-assertions = false
panic = "abort"
codegen-units = 1
lto = true

[profile.release-with-logs]
inherits = "release"
debug-assertions = true
```

Each contract crate uses `crate-type = ["cdylib", "rlib"]`, depends on `soroban-sdk` and the two shared crates, and has `[dev-dependencies] soroban-sdk = { workspace = true, features = ["testutils"] }`.

`overflow-checks = true` is mandatory. Every arithmetic path on amounts must also use `checked_*` and map overflow to `Error::ArithmeticOverflow`.

## 3.4 Root package scripts

| Script | Does |
|---|---|
| `pnpm contracts:build` | `stellar contract build` for all three contracts |
| `pnpm contracts:test` | `cargo test --workspace` |
| `pnpm deploy:testnet` | `tsx scripts/deploy.ts --network testnet` |
| `pnpm seed:testnet` | `tsx scripts/seed.ts --network testnet` |
| `pnpm bindings` | `bash scripts/bindings.sh testnet` |
| `pnpm dev` | `docker compose up` for Postgres plus `tsx watch` for each service |
| `pnpm db:migrate` | Apply `db/migrations/*.sql` in order |
| `pnpm demo:breach` | `tsx scripts/demo/breach.ts --network testnet` |
| `pnpm demo:challenge` | `tsx scripts/demo/challenge.ts --network testnet` |
| `pnpm test` | Vitest across packages and services |

## 3.5 Environment variables

| Variable | Used by | Example |
|---|---|---|
| `STELLAR_NETWORK` | all | `testnet` |
| `STELLAR_RPC_URL` | all | `https://soroban-testnet.stellar.org` |
| `HORIZON_URL` | observer | `https://horizon-testnet.stellar.org` (fallback t0 lookup) |
| `NETWORK_PASSPHRASE` | all | `Test SDF Network ; September 2015` |
| `DATABASE_URL` | observer, indexer, api | `postgres://vaeltrix:vaeltrix@localhost:5432/vaeltrix` |

| Variable | Used by | Example |
|---|---|---|
| `OBSERVER_SECRET` | observer | Stellar secret seed of the registered observer account |
| `KEEPER_SECRET` | observer | May equal `OBSERVER_SECRET`; pays fees for `resolve` and `extend_ttl` |
| `MOCK_ANCHOR_SIGNER_SECRET` | mock-anchor | ed25519 seed registered as a provider signer |
| `MOCK_ANCHOR_PAYOUT_DELAY_MS` | mock-anchor | `150000` to force a breach on template 901 |
| `DEPLOYMENTS_FILE` | all | `deployments/testnet.json` |

Secrets never go in the repo. `.env.example` lists every key with a placeholder.

### 3.6 Continuous integration

One GitHub Actions workflow, three jobs, all required on `main`:

1. **rust:** install toolchain and `wasm32v1-none`, `cargo fmt --check`, `cargo clippy --workspace --all-targets -- -D warnings`, `cargo test --workspace`, `stellar contract build`, upload the three wasm files as artifacts.
2. **ts:** `pnpm install --frozen-lockfile`, `pnpm -r build`, `pnpm -r lint`, `pnpm test`.
3. **vectors:** runs the Rust and TypeScript test-vector suites (§4.6) and fails if either disagrees with `test-vectors/*.json`.

Branches follow `type/short-description` (`feat/bond-claims`, `fix/registry-ttl`, `chore/ci-cache`). Squash merge. Conventional commit titles.

## 4. Shared crates: vaeltrix-types and vaeltrix-verdict

Two small crates are the single source of truth: `vaeltrix-types` holds every on-chain type, error and cross-contract client trait, and `vaeltrix-verdict` holds every rule that decides breach, penalty and bond so contracts and observers can never compute different answers.

Freeze this section before any contract work starts. Changing a field later means regenerating bindings, migrating the database and touching every service.

## 4.1 vaeltrix-types: enums

```
#![no_std]
use soroban_sdk::{contracttype, contracterror, Address, BytesN, String,
Symbol, Vec};

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum ProviderStatus { Active, Suspended, Exited }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum Metric { TimeToCompletion }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum EventKind {
    SenderDebitConfirmed, OnChainSettled, ReceiverAccepted,
    ReceiverCreditConfirmed, RecipientAcknowledged,
}

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum Direction { Inbound, Outbound }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum CarveOut {
    ReceiverBankHoliday, ComplianceHoldDeclared,
    InvalidBeneficiary, SenderInfoIncomplete,
}

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum EvidenceMode { DualSigned, ProviderSigned, ObserverOnly }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum TemplateStatus { Proposed, Active, Deprecated }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum CommitmentStatus { Active, Paused, Exiting, Exited }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum ClaimStatus { Filed, Challenged, Paid, Rejected, Expired }   // D10

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
pub enum BreachKind { LateCompletion, NoCompletion }

#[contracttype] #[derive(Clone, Copy, Debug, Eq, PartialEq)]
```

```
pub enum ClaimOutcome { Upheld, Rejected, Expired }   // emitted in claim_res

// Soroban enums cannot carry named fields, so each penalty variant wraps a
struct.
#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct PerBreachBps { pub bps: u32, pub cap: i128 }

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub enum PenaltyModel {
    PerBreachBps(PerBreachBps),   // V0, the only variant contracts accept
    // V1+: Flat(Flat), Graduated(Graduated), Statistical(Statistical)
}
```

Adding a future penalty variant is a new enum arm. Contracts MUST reject unknown variants with `Error::UnsupportedPenaltyModel`, so old deployments fail closed.

## 4.2 vaeltrix-types: structs

Fields marked NEW are additions to the spec structs; each one traces to a decision in §2.

```
#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Provider {
    pub id: BytesN<32>,              // D4
    pub admin: Address,
    pub signers: Vec<BytesN<32>>,    // ed25519 public keys, max 5
    pub home_domain: String,         // lowercase, no scheme, max 253 bytes
    pub metadata_uri: String,        // max 256 bytes
    pub verified: bool,              // NEW: set by verify_domain
    pub status: ProviderStatus,
    pub registered_at: u64,          // seconds
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Scope {
    pub asset: Address,                      // SAC address of payment asset
    pub direction: Direction,
    pub sender_country: Option<Symbol>,      // ISO 3166-1 alpha-2, e.g. US
    pub receiver_country: Option<Symbol>,
    pub min_amount: i128,
    pub max_amount: i128,
    pub payout_rail: Option<Symbol>,         // BANK | MOBILE_MONEY
}
```

```
#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct SlaTemplate {
    pub id: u32,
    pub version: u32,
    pub metric: Metric,
    pub start_event: EventKind,
    pub end_event: EventKind,
    pub target_ms: u64,
    pub grace_ms: u64,
    pub penalty: PenaltyModel,
    pub scope: Scope,
    pub carve_outs: Vec<CarveOut>,
    pub max_hold_ms: u64,             // NEW: compliance hold cap,
172_800_000 (48h)
    pub claim_window_secs: u64,
    pub evidence_mode: EvidenceMode,
    pub status: TemplateStatus,       // NEW
    pub activates_at: u64,            // NEW: proposed_at + timelock_secs
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Commitment {
    pub id: u64,
    pub provider_id: BytesN<32>,
    pub template_id: u32,
    pub template_version: u32,        // NEW: commitments stay on the version
signed
    pub stake_asset: Address,
    pub stake_total: i128,
    pub stake_reserved: i128,
    pub min_stake: i128,              // NEW: computed at commit, §4.5
    pub max_volume_per_period: i128,  // 0 = no ceiling; enforced off chain
in V0
    pub period_secs: u64,
    pub effective_from: u64,          // seconds
    pub effective_to: Option<u64>,
    pub paused_at: Option<u64>,       // NEW: D6
    pub resumed_at: Option<u64>,      // NEW: D6
    pub exit_requested_at: Option<u64>,
    pub exit_cooldown_secs: u64,      // NEW: frozen at commit, §4.5
    pub open_claims: u32,             // NEW: count of Filed + Challenged
    pub status: CommitmentStatus,
}
```

```
#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct PaymentRef {
    pub payment_id: BytesN<32>,
    pub sep31_tx_id: String,          // NEW: lets the contract recompute
payment_id (D4)
    pub commitment_id: u64,
    pub amount: i128,
    pub t0_ms: u64,
    pub t1_ms: Option<u64>,
    pub pause_ms: u64,                // NEW: total carve-out pause applied
(D2)
    pub evidence_hash: BytesN<32>,    // JCS sha256 of evidence bundle (D3)
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct CompletionReceipt {        // NEW: Path B only (D2)
    pub signer: BytesN<32>,           // must be in provider.signers
    pub signature: BytesN<64>,        // ed25519 over §4.4 layout
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Claim {
    pub id: u64,
    pub commitment_id: u64,
    pub provider_id: BytesN<32>,
    pub payment: PaymentRef,
    pub breach: BreachKind,                       // NEW
    pub claimant: Address,
    pub beneficiary: Address,
    pub bond_payer: Address,                      // NEW: claimant or observer
(D7)
    pub filed_by_observer: Option<Address>,       // NEW: Path A
    pub penalty_amount: i128,
    pub reserved_amount: i128,                    // NEW: min(available,
penalty) (D5)
    pub shortfall: i128,                          // NEW
    pub claim_bond: i128,
    pub status: ClaimStatus,
    pub filed_at: u64,
    pub challenge_deadline: u64,
    pub attestation_deadline: u64,                // NEW: 0 until challenged
    pub counter_evidence_hash: Option<BytesN<32>>, // NEW
```

```
    pub votes_for: u32,
    pub votes_against: u32,
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Observer {
    pub address: Address,
    pub key: BytesN<32>,          // ed25519 key that signs off-chain
attestation docs
    pub active: bool,
    pub added_at: u64,
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct Params {                // NEW: all tunables in one place (D12,
D16)
    pub timelock_secs: u64,
    pub challenge_window_secs: u64,
    pub attestation_window_secs: u64,
    pub exit_extra_secs: u64,          // spec: +7 days
    pub quorum: u32,                   // V0 = 1
    pub min_claim_bond: i128,          // 10_000_000
    pub claim_bond_bps: u32,           // 1_000
    pub rejected_bond_provider_bps: u32, // 5_000
    pub stake_floor: i128,             // 10_000_000_000 (1,000 USDC)
    pub stake_k: u32,                  // 3
    pub max_stake_per_commitment: i128, // 0 = uncapped; testnet cap
recommended
    pub allowed_stake_assets: Vec<Address>,
}

#[contracttype] #[derive(Clone, Debug, Default, Eq, PartialEq)]
pub struct PeriodStats {
    pub payments: u32, pub on_time: u32, pub breaches: u32, pub carved_out:
u32,
    pub p50_ms: u64, pub p95_ms: u64, pub p99_ms: u64,
    pub volume: i128, pub penalties_paid: i128,
}

#[contracttype] #[derive(Clone, Debug, Default, Eq, PartialEq)]
pub struct LifetimeStats {
    pub payments: u64, pub on_time: u64, pub breaches: u64, pub carved_out:
u64,
```

```
    pub volume: i128, pub penalties_paid: i128,
    pub claims_upheld: u32, pub claims_rejected: u32, pub claims_expired:
u32,
    pub shortfall_total: i128,
}

#[contracttype] #[derive(Clone, Debug, Eq, PartialEq)]
pub struct BatchRecord {
    pub observer: Address,
    pub merkle_root: BytesN<32>,
    pub attestation_hash: BytesN<32>,  // D9
    pub stats: PeriodStats,
    pub anchored_at: u64,
}
```

## 4.3 vaeltrix-types: errors and client traits

Codes 1 to 14 match spec §10.6 exactly so external tools stay compatible; 15 and up are new.

```
#[contracterror] #[derive(Copy, Clone, Debug, Eq, PartialEq, PartialOrd,
Ord)]
#[repr(u32)]
pub enum Error {
    NotAuthorized = 1, ProviderNotActive = 2, TemplateNotActive = 3,
    InsufficientStake = 4, OutOfScope = 5, ClaimWindowClosed = 6,
    DuplicateClaim = 7, InvalidEvidence = 8, NotInBreach = 9,
    ChallengeWindowOpen = 10, OpenClaimsExist = 11, CooldownActive = 12,
    UnknownObserver = 13, AlreadyAttested = 14,
    // new in this guide
    AlreadyInitialized = 15, NotInitialized = 16, InvalidParams = 17,
    UnknownProvider = 18, UnknownTemplate = 19, UnknownCommitment = 20,
    UnknownClaim = 21, ProviderNotVerified = 22, ProviderExists = 23,
    TemplateExists = 24, TimelockActive = 25, AssetNotAllowed = 26,
    AssetMismatch = 27, AmountOutOfRange = 28, NotCovered = 29,
    CommitmentNotActive = 30, ClaimNotOpen = 31, AttestationWindowClosed =
32,
    QuorumNotReached = 33, BelowMinStake = 34, StakeCapExceeded = 35,
    PaymentIdMismatch = 36, UnknownSigner = 37, TooManySigners = 38,
```

```
    UnsupportedPenaltyModel = 39, ArithmeticOverflow = 40, BondAlreadySet =
41,
    BatchExists = 42, InvalidPeriod = 43, NothingPending = 44,
}
```

Cross-contract client traits live here so bond and scoreboard can call registry without importing its wasm:

```
#[soroban_sdk::contractclient(name = "RegistryClient")]
pub trait RegistryApi {
    fn get_provider(env: Env, provider_id: BytesN<32>) -> Provider;
    fn get_template_version(env: Env, template_id: u32, version: u32) ->
SlaTemplate;
    fn is_active_observer(env: Env, observer: Address) -> bool;
    fn active_observer_count(env: Env) -> u32;
    fn params(env: Env) -> Params;
}

#[soroban_sdk::contractclient(name = "ScoreboardClient")]
pub trait ScoreboardApi {
    fn record_claim_outcome(env: Env, claim: Claim, outcome: ClaimOutcome);
}
```

`Params` live in the registry and are read by bond on each call that needs them. One place to change, one timelock.

## 4.4 Byte layout for on-chain-verified receipts

Path B (§2, D2) needs a message both TypeScript and Rust can build byte for byte. Fixed width, big-endian, no padding:

| Offset | Length | Field | Encoding |
|---|---|---|---|
| 0 | 19 | domain tag | ASCII `VAELTRIX:RECEIPT:v1` |
| 19 | 32 | `network_id` | `sha256(network_passphrase)`; on chain `env.ledger().network_id()` |
| 51 | 32 | `payment_id` | raw bytes |
| 83 | 8 | `commitment_id` | u64 BE |
| 91 | 16 | `amount` | i128 BE, two's complement |

| Offset | Length | Field | Encoding |
|---|---|---|---|
| 107 | 8 | `t0_ms` | u64 BE |
| 115 | 8 | `t1_ms` | u64 BE |
| 123 | 8 | `pause_ms` | u64 BE |
| total | 131 | | |

The provider signs these 131 bytes directly with ed25519 (no pre-hash). `env.crypto().ed25519_verify` panics on a bad signature rather than returning an error; the contract MUST check `signer` membership first and return `Error::UnknownSigner` so the common failure has a clean code.

## 4.5 vaeltrix-verdict: the pure rules

A `#![no_std]` crate with no Soroban dependency, primitive types only, so it can be unit tested in plain Rust and ported line for line to TypeScript in `@vaeltrix/core`.

```
pub enum Verdict { Pending, OnTime { latency_ms: u64 }, Late { latency_ms:
u64 }, NoCompletion }

/// Deadline after which a payment is in breach. pause_ms extends it.
pub fn deadline_ms(t0_ms: u64, target_ms: u64, grace_ms: u64, pause_ms: u64)
-> Option<u64>;

/// latency = t1 - t0 - pause. Late iff latency > target + grace.
/// No t1: NoCompletion iff now > deadline, else Pending. t1 < t0 is invalid
(None).
pub fn evaluate(t0_ms: u64, t1_ms: Option<u64>, now_ms: u64,
                target_ms: u64, grace_ms: u64, pause_ms: u64) ->
Option<Verdict>;

/// floor(amount * bps / 10_000), capped. Rounds in the provider's favour.
pub fn penalty(amount: i128, bps: u32, cap: i128) -> Option<i128>;

/// Largest penalty any in-scope payment can produce: penalty(max_amount,
bps, cap).
pub fn max_penalty(max_amount: i128, bps: u32, cap: i128) -> Option<i128>;

/// max(min_bond, penalty * bond_bps / 10_000)
pub fn claim_bond(penalty: i128, min_bond: i128, bond_bps: u32) ->
```

```
Option<i128>;

/// max(floor, k * daily_cap * bps / 10_000). daily_cap = 0 means use floor.
pub fn min_stake(floor: i128, k: u32, daily_cap: i128, bps: u32) ->
Option<i128>;

/// claim_window + challenge + attestation + extra. Spec §12.2 exit cooldown.
pub fn exit_cooldown_secs(claim_window: u64, challenge: u64, attestation:
u64, extra: u64) -> Option<u64>;

/// Last moment a claim may be filed: deadline_ms + claim_window_secs * 1000.
pub fn claim_deadline_ms(deadline_ms: u64, claim_window_secs: u64) ->
Option<u64>;

/// Rejected bond split (D13). Returns (to_provider, per_observer,
dust_to_provider).
pub fn split_rejected_bond(bond: i128, provider_bps: u32, observers: u32) ->
Option<(i128, i128, i128)>;
```

Every function returns `None` on overflow or invalid input; callers map `None` to `Error::ArithmeticOverflow` or `Error::InvalidEvidence`. Nothing in this crate panics.

Carve-out rules are applied off chain by the observer, which reports the resulting `pause_ms`. The verdict crate still exposes `fn apply_carve_outs(events: &[CarveOutSpan], max_hold_ms: u64) -> CarveOutResult` so observers in every language share the logic: overlapping spans merge, a compliance hold contributes at most `max_hold_ms`, and an `InvalidBeneficiary` span marks the payment out of scope.

## 4.6 Test vectors

`test-vectors/verdict.json` is an array of cases. Rust ( `crates/vaeltrix-verdict/tests/vectors.rs` ) and TypeScript ( `packages/core/test/vectors.test.ts` ) both load the same file and assert identical outputs.

```
{
  "name": "late by one ms past grace",
  "fn": "evaluate",
  "input": { "t0_ms": 1760000000000, "t1_ms": 1760003720001, "now_ms":
1760010000000,
             "target_ms": 3600000, "grace_ms": 120000, "pause_ms": 0 },
  "expect": { "verdict": "Late", "latency_ms": 3720001 }
}
```

Minimum required cases, at least 40 in total:

- [ ] exactly on target, exactly on target plus grace (on time), one ms over (late)
- [ ] no t1 before deadline (pending), at deadline (pending), one ms after (no completion)
- [ ] t1 earlier than t0 (invalid)
- [ ] pause extends the deadline; pause larger than latency
- [ ] penalty below cap, exactly at cap, above cap, amount 1 stroop (rounds to 0), amount at i128 overflow edge
- [ ] claim bond at min floor and above it
- [ ] min stake with spec numbers: floor 1,000 USDC, k 3, cap 100,000 USDC, 50 bps gives 1,500 USDC
- [ ] overlapping carve-out spans, compliance hold beyond 48h capped
- [ ] rejected bond split with 1, 3 and 5 observers, odd stroops

`test-vectors/ids.json` holds at least five `provider_id` and `payment_id` derivations, and `receipts.json` holds at least three Path B messages with a fixed test keypair, the 131 bytes as hex and the signature.

# 5. Contract: vaeltrix_registry

The registry owns identity and configuration: providers, templates, observers, protocol parameters and the admin timelock. It holds no funds and calls no other contract.

## 5.1 Storage

```
#[contracttype]
pub enum DataKey {
    Admin,                      // Instance: Address
    PendingAdmin,               // Instance: Option<Address>
    Params,                     // Instance: Params
    Verifier,                   // Instance: Address allowed to call
verify_domain (default admin)
    NextActionId,               // Instance: u32
    Provider(BytesN<32>),       // Persistent: Provider
    ProviderByAdmin(Address),   // Persistent: Vec<BytesN<32>>
    Template(u32, u32),         // Persistent: SlaTemplate keyed (id,
version)
    TemplateLatest(u32),        // Persistent: u32 latest version number
    TemplateIds,                // Persistent: Vec<u32>
```

```
    Observer(Address),           // Persistent: Observer
    ObserverList,                // Persistent: Vec<Address> (bounded, max
32)
    Action(u32),                 // Persistent: PendingAction
}

#[contracttype]
pub enum ActionKind {
    SetParams(Params),
    AddObserver(Address, BytesN<32>),
    RemoveObserver(Address),
    SetVerifier(Address),
    Upgrade(BytesN<32>),
}

#[contracttype]
pub struct PendingAction { pub id: u32, pub kind: ActionKind, pub eta: u64,
pub executed: bool, pub cancelled: bool }
```

Every public function extends instance TTL. Every read or write of a persistent key extends that key's TTL (§8.3).

## 5.2 Construction and admin

| Function | Auth | Behaviour | Errors | Event |
|---|---|---|---|---|
| `__constructor(admin: Address, params: Params)` | none (deploy-time) | Validates params (§5.6), stores admin, params, verifier = admin, next action id 1. | `InvalidParams` | `init` |
| `propose_admin(new_admin: Address)` | admin | Stores pending admin. Two-step so a typo cannot brick the contract. | | `adm_prop` |
| `accept_admin()` | pending admin | Moves pending to admin. | `NothingPending` | `adm_set` |
| `propose(kind: ActionKind) -> u32` | admin | Stores action with `eta = now + params.timelock_secs`. For `SetParams`, validates first. | `InvalidParams` | `act_prop` (id, eta) |
| `execute(action_id: u32)` | anyone | Requires `now >= eta`, not executed, not cancelled. Applies the action. `Upgrade` calls `env.deployer().update_current_contract_wasm(hash)`. | `TimelockActive`, `NothingPending` | `act_exec` |
| `cancel(action_id: u32)` | admin | Marks cancelled. | `NothingPending` | `act_canc` |

| Function | Auth | Behaviour | Errors | Event |
|---|---|---|---|---|
| `admin() -> Address`, `params() -> Params`, `action(id) -> PendingAction` | none | Views. | | |

On testnet `timelock_secs = 60`, so the propose and execute pattern costs one minute and keeps the mainnet code path exercised.

## 5.3 Providers

**`register_provider(admin: Address, home_domain: String, signers: Vec<BytesN<32>>, metadata_uri: String) -> BytesN<32>`**

1. `admin.require_auth()`.
2. Validate `home_domain`: 1 to 253 bytes, only `a-z 0-9 . -`, no scheme, no trailing dot. Reject uppercase rather than lowercasing silently (`InvalidParams`).
3. Validate `signers`: 1 to 5 entries, no duplicates (`TooManySigners`, `InvalidParams`).
4. Validate `metadata_uri`: at most 256 bytes.
5. `id = provider_id(home_domain, admin)` per §2 D4. If `Provider(id)` exists: `ProviderExists`.
6. Store with `verified = false`, `status = Active`, `registered_at = now`. Append to `ProviderByAdmin(admin)`.
7. Emit `prov_reg` with `(id, admin, home_domain)`.

**`update_signers(provider_id, signers)`**: provider admin auth, same validation as step 3, emit `prov_sig`. Signer rotation takes effect immediately; receipts signed by a removed key stop verifying, which is the point.

**`update_metadata(provider_id, metadata_uri)`**: provider admin auth, emit `prov_meta`.

**`verify_domain(provider_id)`**: `verifier.require_auth()`. Sets `verified = true`, emits `prov_ver`. Off chain, the verifier script (VX-207) fetches `https://<home_domain>/.well-known/stellar.toml`, reads the `[VAELTRIX]` block (spec §17.1) and only calls this when `PROVIDER_ID` and `REGISTRY_CONTRACT` match.

**`unverify_domain(provider_id)`**: verifier auth, sets `verified = false`, emits `prov_unv`. Used when the toml stops matching.

**`suspend_provider(provider_id, reason: Symbol)`** and **`reinstate_provider(provider_id)`**: admin auth. Suspension blocks new commitments and new stake top-ups only; it never touches open claims or stake. Emit `prov_sus` and `prov_rein`.

**Views:** `get_provider(id) -> Provider`, `providers_by_admin(admin) -> Vec<BytesN<32>>`, `provider_id_for(home_domain, admin) -> BytesN<32>` (pure helper the SDK uses to check its own derivation).

## 5.4 Templates

**`propose_template(template: SlaTemplate) -> u32`** (admin auth)

1. If `TemplateLatest(template.id)` exists, `template.version` MUST equal latest plus 1, otherwise `version` MUST be 1 (`TemplateExists`).
2. Validate (`InvalidParams` unless stated):
   - `metric == TimeToCompletion`, `start_event == OnChainSettled`, `end_event == ReceiverCreditConfirmed` (V0 only supports this pair).
   - `target_ms > 0`, `grace_ms <= target_ms`.
   - `penalty` is `PerBreachBps` with `1 <= bps <= 10_000` and `cap > 0` (`UnsupportedPenaltyModel` for other variants).
   - `scope.min_amount > 0`, `scope.max_amount >= scope.min_amount`.
   - `scope.asset` is in `params.allowed_stake_assets` (`AssetNotAllowed`).
   - `claim_window_secs > 0`, `max_hold_ms <= 172_800_000`.
   - `carve_outs` has no duplicates.
3. Force `status = Proposed`, `activates_at = now + params.timelock_secs`, whatever the caller passed.
4. Store `Template(id, version)`, update `TemplateLatest`, append id to `TemplateIds` if new.
5. Emit `tmpl_prop` with `(id, version, activates_at)`. Return `version`.

**`activate_template(id, version)`**: anyone, requires `now >= activates_at` and `status == Proposed`. Sets `Active`, emits `tmpl_act`. Permissionless because the admin already approved it at proposal; the timelock is the review window.

**`cancel_template(id, version)`**: admin, only while `Proposed`. Sets `Deprecated`. Emits `tmpl_canc`.

**`deprecate_template(id, version)`**: admin. Sets `Deprecated`, emits `tmpl_dep`. Existing commitments keep working on the version they signed; new `commit` calls are rejected.

**Views:** `get_template(id) -> SlaTemplate` (latest Active version, else `TemplateNotActive`), `get_template_version(id, version) -> SlaTemplate`, `template_ids() -> Vec<u32>`.

Templates are immutable once stored. There is no update function, by design (spec §8.4).

## 5.5 Observers

Observers are only added and removed through `propose(AddObserver(..))` and `propose(RemoveObserver(..))` then `execute`. Execution of add stores `Observer { active: true }` and appends to `ObserverList` (max 32, `InvalidParams` beyond). Removal sets `active = false` and keeps the record so past attestations stay attributable.

**Views:** `is_active_observer(addr) -> bool`, `active_observer_count() -> u32`, `observers() -> Vec<Observer>`.

A provider admin MUST NOT be an observer (spec §13.4 independence). `execute` of `AddObserver` checks `ProviderByAdmin(observer)` is empty and fails with `InvalidParams` otherwise.

## 5.6 Params validation

`SetParams` and the constructor reject unless all hold:

- `challenge_window_secs >= 60`, `attestation_window_secs >= 60`
- `1 <= quorum <= 32`
- `min_claim_bond > 0`, `claim_bond_bps <= 10_000`, `rejected_bond_provider_bps <= 10_000`
- `stake_floor > 0`, `stake_k >= 1`, `max_stake_per_commitment >= 0`
- `allowed_stake_assets` non-empty, at most 8, no duplicates

Changes to params never apply retroactively to an existing commitment's `exit_cooldown_secs` or `min_stake`; those are frozen at commit (§6).

### 5.7 Recommended testnet params

| Field | Testnet value | Mainnet pilot value |
|---|---|---|
| `timelock_secs` | 60 | 1,209,600 (14 days, longer than every exit cooldown) |
| `challenge_window_secs` | 180 | 172,800 (48h) |
| `attestation_window_secs` | 300 | 259,200 (72h) |
| `exit_extra_secs` | 120 | 604,800 (7 days) |
| `quorum` | 1 | 3 |
| `min_claim_bond` | 10,000,000 (1 USDC) | same |
| `claim_bond_bps` | 1,000 | same |
| `rejected_bond_provider_bps` | 5,000 | same |
| `stake_floor` | 1,000,000,000 (100 USDC) | 10,000,000,000 (1,000 USDC) |
| `stake_k` | 3 | 3 |
| `max_stake_per_commitment` | 100,000,000,000 (10,000 USDC) | set by governance |
| `allowed_stake_assets` | [test USDC SAC] | [Circle USDC SAC] |

## 6. Contract: vaeltrix_bond

The bond contract holds every token in the protocol: commitments, stake, claims, bonds and payouts live together so a slash is one internal state change plus one token transfer, never a cross-contract hop that can fail halfway.

## 6.1 Storage

```
#[contracttype]
pub enum DataKey {
    Admin,                          // Instance: Address
    Registry,                       // Instance: Address
    Scoreboard,                     // Instance: Address
    NextCommitmentId,               // Instance: u64, starts at 1
    NextClaimId,                    // Instance: u64, starts at 1
    PendingUpgrade,                 // Instance: Option<(BytesN<32>, u64
eta)>
    Commitment(u64),                // Persistent: Commitment
    CommitmentsByProvider(BytesN<32>), // Persistent: Vec<u64>
    Claim(u64),                     // Persistent while open, then TTL ~90
days
    ClaimByPayment(BytesN<32>),     // Persistent: u64, kept at least claim
window + 90 days
    ClaimVotes(u64),                // Persistent: Vec<(Address, bool)>
(bounded by observer count)
}

pub const MAX_PAUSE_MS: u64 = 604_800_000;   // 7 days, ceiling on pause_ms
in any claim
```

## 6.2 Construction and upgrade

- `__constructor(admin: Address, registry: Address, scoreboard: Address)`: stores the three addresses, counters at 1.
- `propose_upgrade(wasm_hash: BytesN<32>)`: admin auth, `eta = now + registry.params().timelock_secs`, event `upg_prop`.
- `execute_upgrade()`: anyone after eta, calls `update_current_contract_wasm`, event `upg_exec`.
- `cancel_upgrade()`: admin auth.

All other admin powers are deliberately absent. The admin cannot move stake, cancel a claim or change an outcome (spec §4.4).

## 6.3 Coverage rule

A payment with start `t0_s = t0_ms / 1000` is covered by commitment `c` when all of these hold:

1. `t0_s >= c.effective_from`
2. `c.effective_to` is `None` or `t0_s < effective_to`
3. `c.exit_requested_at` is `None` or `t0_s < exit_requested_at`
4. Not inside the latest pause: if `paused_at = Some(p)` then either `resumed_at = Some(r)` with `r > p` and `t0_s` not in `[p, r)`, or `resumed_at` is `None` and `t0_s < p`.

Implement as a private `fn is_covered(c: &Commitment, t0_ms: u64) -> bool` and expose it as the view `is_covered(commitment_id, t0_ms)` so the observer and SDK use the same rule.

## 6.4 Exit cooldown (correction to spec §12.2)

The spec sets cooldown to claim plus challenge plus attestation windows plus 7 days, counted from the exit request. That misses payments started just before the exit request: their breach deadline lands up to `target + grace + pause` later, and their claim window only starts then. The corrected value, frozen into the commitment at `commit` time:

```
\text{cooldown} = \left\lceil \frac{\text{target\_ms} + \text{grace\_ms} +
\text{MAX\_PAUSE\_MS}}{1000} \right\rceil + \text{claim\_window} +
\text{challenge} + \text{attestation} + \text{exit\_extra}
```

Call `verdict::exit_cooldown_secs` with the first argument set to `claim_window_secs + ceil((target_ms + grace_ms + MAX_PAUSE_MS) / 1000)`.

## 6.5 Commitment functions

**`commit(provider_id, template_id, stake_asset, amount, max_volume_per_period, period_secs) -> u64`**

1. `provider = registry.get_provider(provider_id)`; `provider.admin.require_auth()`.
2. Provider `status == Active` (`ProviderNotActive`) and `verified` (`ProviderNotVerified`).
3. `template = registry.get_template(template_id)` (latest Active version, `TemplateNotActive`). Store its version in the commitment.
4. `stake_asset == template.scope.asset` (`AssetMismatch`, D8) and in `params.allowed_stake_assets` (`AssetNotAllowed`).
5. `period_secs > 0` when `max_volume_per_period > 0` (`InvalidParams`). Derive `daily_cap = max_volume_per_period * 86_400 / period_secs` or 0.

6. `min_stake = verdict::min_stake(params.stake_floor, params.stake_k, daily_cap, bps)`; `amount >= min_stake` (`BelowMinStake`) and `amount >= max_penalty(template)` (`InsufficientStake`).
7. `params.max_stake_per_commitment == 0` or `amount <= max_stake_per_commitment` (`StakeCapExceeded`).
8. Compute and freeze `exit_cooldown_secs` (§6.4).
9. Token transfer `provider.admin -> contract` for `amount` via `token::Client::new(&env, &stake_asset).transfer(..)`.
10. Store `Commitment { status: Active, effective_from: now, stake_total: amount, stake_reserved: 0, open_claims: 0, .. }`. Append to `CommitmentsByProvider`.
11. Emit `commit (commitment_id, provider_id, template_id, amount)`. Return the id.

| Function | Auth | Allowed from status | Effect | Errors | Event |
|---|---|---|---|---|---|
| `add_stake(id, amount)` | provider admin | Active, Paused | Transfer in, `stake_total += amount`, cap check | `CommitmentNotActive`, `ProviderNotActive`, `StakeCapExceeded` | `stake_chg (id, +amount, total)` |
| `withdraw_excess(id, amount)` | provider admin | Active, Paused | Requires `stake_total - amount - stake_reserved >= min_stake`. Transfer out to admin. | `InsufficientStake` | `stake_chg (id, -amount, total)` |
| `pause(id)` | provider admin | Active | `status = Paused`, `paused_at = now`, `resumed_at = None` | `CommitmentNotActive` | `cmt_pause` |
| `resume(id)` | provider admin | Paused | `status = Active`, `resumed_at = now` | `CommitmentNotActive` | `cmt_resum` |
| `request_exit(id)` | provider admin | Active, Paused | `status = Exiting`, `exit_requested_at = now` | `CommitmentNotActive` | `exit_req` |

| Function | Auth | Allowed from status | Effect | Errors | Event |
|---|---|---|---|---|---|
| `finalize_exit(id)` | provider admin | Exiting | Requires `now >= exit_requested_at + exit_cooldown_secs` and `open_claims == 0`. Transfers `stake_total` to admin, sets it to 0, `status = Exited`. | `CooldownActive`, `OpenClaimsExist` | `exit_fin` |

An exiting commitment still pays claims for payments it covered. That is the whole reason for the cooldown.

## 6.6 Claim filing

**`file_claim(claimant: Address, beneficiary: Address, payment: PaymentRef, observer: Option<Address>, receipt: Option<CompletionReceipt>) -> u64`**

Replaces the spec's `evidence: Bytes` argument (D2, D3). Checks run in this order so the cheapest failures come first:

1. `claimant.require_auth()`.
2. Exactly one of `observer`, `receipt` is `Some`, else `InvalidEvidence`.
3. Path A: `observer.require_auth()` and `registry.is_active_observer(observer)` (`UnknownObserver`). When the observer files for a client, `claimant == observer`, so one signature covers both.
4. Load commitment (`UnknownCommitment`), provider, and `registry.get_template_version(c.template_id, c.template_version)`.
5. `payment.payment_id == payment_id(payment.sep31_tx_id, c.provider_id)` (`PaymentIdMismatch`).
6. `ClaimByPayment(payment_id)` absent (`DuplicateClaim`).
7. `scope.min_amount <= amount <= scope.max_amount` (`OutOfScope`).
8. `is_covered(c, payment.t0_ms)` (`NotCovered`).
9. `payment.pause_ms <= MAX_PAUSE_MS` (`InvalidEvidence`).
10. `now_ms = ledger.timestamp() * 1000`; `verdict::evaluate(..)` returns `Late` or `NoCompletion`, else `NotInBreach`. Path B additionally requires `Late`.

11. Path B: `receipt.signer` in `provider.signers` (`UnknownSigner`), build the §4.4 message, `env.crypto().ed25519_verify(&signer, &msg, &signature)`.
12. `now_ms <= claim_deadline_ms` (`ClaimWindowClosed`).
13. `penalty = verdict::penalty(amount, bps, cap)`; zero penalty is `OutOfScope`.
14. `bond = verdict::claim_bond(penalty, params.min_claim_bond, params.claim_bond_bps)`; transfer `bond` from claimant to contract.
15. `available = stake_total - stake_reserved`; `reserved = min(available, penalty)`; `shortfall = penalty - reserved` (D5). `stake_reserved += reserved`; `open_claims += 1`.
16. Store claim with `status = Filed`, `challenge_deadline = now + params.challenge_window_secs`, `attestation_deadline = 0`, `bond_payer = claimant`, `filed_by_observer = observer`. Store `ClaimByPayment(payment_id) = id`.
17. Emit `claim_new (claim_id, payment_id, penalty, reserved, shortfall)`. Return the id.

Filing is allowed against an `Exiting` commitment and a suspended provider. It is not allowed against `Exited` (nothing left to pay from), which `NotCovered` handles because `exit_requested_at` precedes t0 or the stake is gone.

### 6.7 Challenge, attest, resolve

**`challenge(claim_id, counter_evidence_hash: BytesN<32>)`**: provider admin auth. Requires `status == Filed` and `now < challenge_deadline` (`ClaimNotOpen`). Sets `Challenged`, stores the hash, `attestation_deadline = now + params.attestation_window_secs`. Free to the provider. Emits `claim_chal`.

**`attest(observer, claim_id, uphold: bool, attestation_hash: BytesN<32>)`**: `observer.require_auth()`, active observer (`UnknownObserver`), `status == Challenged` (`ClaimNotOpen`), `now <= attestation_deadline` (`AttestationWindowClosed`), not already voted (`AlreadyAttested`). Appends to `ClaimVotes`, bumps `votes_for` or `votes_against`. Emits `claim_att (claim_id, observer, uphold, attestation_hash)`.

An observer that filed the claim may still attest on it in V0 (there is only one). From V1 the filing observer is excluded from voting on its own claim.

**`resolve(claim_id)`**: anyone.

| Current status | Condition | Outcome | Token movements |
|---|---|---|---|
| Filed | `now < challenge_deadline` | error `ChallengeWindowOpen` | none |
| Filed | `now >= challenge_deadline` | Upheld, status `Paid` | `reserved` to beneficiary; bond back to `bond_payer` |
| Challenged | `votes_for >= quorum` | Upheld, status `Paid` | same as above |
| Challenged | `votes_against >= quorum` | Rejected, status `Rejected` | `reserved` released; bond split per D13 |
| Challenged | neither, `now > attestation_deadline` | Expired, status `Expired` | `reserved` released; bond back to `bond_payer` |
| Challenged | neither, before deadline | error `QuorumNotReached` | none |
| Paid, Rejected, Expired | any | error `ClaimNotOpen` | none |

On every successful resolve: update storage first ( `stake_reserved -= reserved` , on Upheld also `stake_total -= reserved` , `open_claims -= 1` , new status), then perform token transfers, then call `scoreboard.record_claim_outcome(claim, outcome)` , then emit `claim_res (claim_id, outcome, amount_paid)` .

When `quorum` is greater than the number of active observers the claim can only expire. The observer service MUST alert on that state.

## 6.8 Maintenance and views

- `extend_commitment_ttl(id)` and `extend_claim_ttl(id)` : permissionless keeper calls (spec §10.5).

- Views: `commitment(id)`, `claim(id)`, `claim_by_payment(payment_id) -> Option<u64>`, `available_stake(id)`, `max_penalty(id)`, `is_covered(id, t0_ms)`, `commitments_by_provider(provider_id)`, `claim_votes(id)`, `registry()`, `scoreboard()`.

## 6.9 Invariants

Every state-changing function MUST preserve all of these. Each one is a named test in §9.

| ID | Invariant | Source |
|---|---|---|
| I1 | `0 <= stake_reserved <= stake_total` for every commitment | spec §10.2 #1 |
| I2 | For every commitment, `stake_reserved` equals the sum of `reserved_amount` over its Filed and Challenged claims | spec #2, made checkable |
| I3 | `finalize_exit` fails while `open_claims > 0` or before `exit_requested_at + exit_cooldown_secs` | spec #3 |
| I4 | `exit_cooldown_secs` is at least the corrected formula in §6.4 | spec #4, corrected |
| I5 | At most one claim per `payment_id`, ever | spec #5 |
| I6 | Tokens leave the contract only to: a beneficiary (upheld), the provider admin (exit, excess withdrawal, rejected-bond share), the bond payer (bond refund), or a voting observer (rejected-bond share) | spec #6, extended for D13 |
| I7 | Contract token balance per asset is at least the sum of `stake_total` over non-Exited commitments plus the sum of `claim_bond` over open claims | new |
| I8 | `open_claims` equals the count of Filed plus Challenged claims for that commitment | new |
| I9 | A claim's status only moves along the transitions in §6.7 | new |

### 6.10 Template definitions to seed

| ID | Name | Asset | Amount range (USDC) | Target | Grace | Penalty | Cap (USDC) | Claim window | Max hold |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Fast inbound bank payout | USDC | 1 to 1,000 | 60 min | 2 min | 50 bps | 50 | 7 days | 48h |
| 2 | Same-day inbound bank payout | USDC | any (1 to 10,000,000) | 8 h | 5 min | 25 bps | 250 | 14 days | 48h |
| 3 | Instant mobile money payout | USDC | 1 to 2,000 | 15 min | 1 min | 100 bps | 20 | 7 days | 48h |
| 901 | Testnet demo | test USDC | 1 to 10,000 | 2 min | 10 s | 500 bps | 100 | 10 min | 1 min |

Template 1 in the spec JSON uses `min_amount 10000000` and `max_amount 10000000000`, which is 1 to 1,000 USDC with 7 decimals, while the spec table says 10 to 10,000 USDC. The JSON values are used above; confirm with the team (§18).

Template 901 uses 500 bps so the demo penalty is visible: a 100 USDC payment that misses by any amount pays 5 USDC.

## 7. Contract: vaeltrix_scoreboard

The scoreboard stores per-period aggregates published by observers and per-provider claim outcomes pushed by the bond contract; it holds no funds, so a bug here can distort rankings but never touch stake.

## 7.1 Storage

```
#[contracttype]
pub enum DataKey {
    Admin,                                  // Instance: Address
    Registry,                               // Instance: Address
    Bond,                                   // Instance: Option<Address>, set
once (D15)
    Batch(u64, u64),                        // Persistent: BatchRecord keyed
(commitment_id, period)
    BatchPeriods(u64),                      // Persistent: Vec<u64> last 90
periods per commitment
    Lifetime(BytesN<32>),                   // Persistent: LifetimeStats per
provider
    CommitmentProvider(u64),                // Persistent: BytesN<32> cache,
filled from anchor_batch input
}
```

## 7.2 Functions

**`__constructor(admin: Address, registry: Address)`**

**`set_bond(bond: Address)`**: admin auth, only once (`BondAlreadySet`). Emits `bond_set`.

**`anchor_batch(observer, commitment_id, provider_id, period_start, period_end, merkle_root, stats: PeriodStats, attestation_hash)`**

1. `observer.require_auth()`; `registry.is_active_observer(observer)` (`UnknownObserver`).
2. `period_start % 86_400 == 0` and `period_end == period_start + 86_400` and `period_end <= now` (`InvalidPeriod`, D14).
3. `Batch(commitment_id, period)` absent (`BatchExists`).
4. Sanity: `on_time + breaches + carved_out == payments`, `p50 <= p95 <= p99`, `volume >= 0`, `penalties_paid >= 0` (`InvalidParams`).
5. Store `BatchRecord`, push `period` into `BatchPeriods` (drop oldest beyond 90).
6. Add `payments`, `on_time`, `breaches`, `carved_out`, `volume` into `Lifetime(provider_id)`. `penalties_paid` is NOT added from batches; it only comes from `record_claim_outcome`, which is fully on chain.
7. Emit `batch (commitment_id, period, merkle_root, observer)`.

The spec signature has no `provider_id`; it is added because the scoreboard must not call bond to look it up (one-way dependency, bond calls scoreboard). In V0 the observer is trusted to pass the right one; V1 checks it against a cached value set by bond on `commit`.

**`record_claim_outcome(claim: Claim, outcome: ClaimOutcome)`**

1. `bond.require_auth()` where `bond` is the stored address. Soroban treats a direct cross-contract call from bond as authorised by bond, so no signature is needed in practice; any other caller fails (`NotAuthorized`).
2. Update `Lifetime(claim.provider_id)`: Upheld adds `claim.reserved_amount` to `penalties_paid` and `claim.shortfall` to `shortfall_total`, increments `claims_upheld`; Rejected increments `claims_rejected`; Expired increments `claims_expired`.
3. Emit `claim_out (claim_id, provider_id, outcome)`.

**Views:** `stats(commitment_id, period) -> PeriodStats`, `batch(commitment_id, period) -> BatchRecord`, `periods(commitment_id) -> Vec<u64>`, `lifetime(provider_id) -> LifetimeStats`, `bond() -> Option<Address>`.

## 7.3 What lives on chain versus off chain

| Metric (spec §14.1) | Where it is computed | Where it is stored |
|---|---|---|
| Tagged payments, on-time, breaches, carve-outs | Observer, from payment records | `Batch` on chain, full records in observer log |
| p50, p95, p99 latency | Observer | `Batch` on chain |
| Penalties paid, claim outcomes, shortfall | Bond contract | `Lifetime` on chain, events |
| Stake at risk | Bond contract | `Commitment` storage |
| Coverage ratio, reliability score | Indexer and API | Postgres only, recomputable |

The reliability score formula (spec §14.3) is deliberately not in any contract so it can change without an upgrade.

### 7.4 Merkle tree definition (shared by observer and anyone verifying)

- Leaf = `sha256(0x00 || JCS(record))` where record is `{payment_id, amount, t0_ms, t1_ms, pause_ms, carve_out, breach}` (amount as decimal string, t1 null when absent).
- Leaves sorted by `payment_id` ascending (bytewise).
- Node = `sha256(0x01 || left || right)`. An odd node at any level is promoted unchanged.
- Empty period: root = 32 zero bytes, and `payments` must be 0.

The 0x00 and 0x01 prefixes prevent second-preimage attacks between leaves and nodes. `@vaeltrix/core` exports `merkleRoot(records)` and `merkleProof(records, paymentId)`.

## 8. Cross-contract wiring, storage TTL, events and auth

Calls only flow one way: bond reads registry and writes scoreboard, scoreboard reads registry, registry calls nobody; keeping that direction is what stops a scoreboard bug from ever reaching stake.

### 8.1 Call graph

| Caller | Callee | Function | Why |
|---|---|---|---|
| bond | registry | `get_provider`, `get_template`, `get_template_version`, `is_active_observer`, `params` | Identity, rules and parameters at call time |
| bond | scoreboard | `record_claim_outcome` | Lifetime claim stats |
| bond | token (SAC) | `transfer` | Stake in and out, bonds, payouts |
| scoreboard | registry | `is_active_observer` | Gate `anchor_batch` |

Use the `contractclient` traits from §4.3 for registry and scoreboard, and `soroban_sdk::token::Client` for the asset. Never use `contractimport!` on a sibling wasm; it couples build order and breaks when a sibling changes.

Registry calls are not free. `file_claim` makes three of them; that is acceptable for V0. If simulation shows resource limits are close, cache `Params` in bond instance storage with a `sync_params()` call (VX-318, stretch).

## 8.2 Authorisation matrix

| Function | Required signer |
|---|---|
| registry: `propose`, `cancel`, `propose_template`, `cancel_template`, `deprecate_template`, `suspend_provider`, `reinstate_provider`, `propose_admin` | registry admin |
| registry: `register_provider` | the `admin` argument |
| registry: `update_signers`, `update_metadata` | provider admin |
| registry: `verify_domain`, `unverify_domain` | verifier |
| registry: `execute`, `activate_template` | anyone (time-gated) |
| bond: `commit`, `add_stake`, `withdraw_excess`, `pause`, `resume`, `request_exit`, `finalize_exit`, `challenge` | provider admin |
| bond: `file_claim` | claimant, plus observer on Path A |
| bond: `attest` | observer |
| bond: `resolve`, `extend_*_ttl`, `execute_upgrade` | anyone |
| bond: `propose_upgrade`, `cancel_upgrade` | bond admin |
| scoreboard: `set_bond` | scoreboard admin |
| scoreboard: `anchor_batch` | observer |
| scoreboard: `record_claim_outcome` | bond contract only |

Tests MUST cover every row with both the right signer (passes) and a wrong signer (fails). Do not rely on `env.mock_all_auths()` for these tests; use `env.mock_auths(&[..])` with the exact expected invocation.

## 8.3 Storage TTL policy

Testnet ledgers close roughly every 5 seconds, so one day is about 17,280 ledgers.

```
pub const DAY_LEDGERS: u32 = 17_280;
pub const INSTANCE_BUMP: u32 = 30 * DAY_LEDGERS;      // extend to 30 days
pub const INSTANCE_THRESHOLD: u32 = 7 * DAY_LEDGERS;  // when below 7 days
pub const PERSIST_BUMP: u32 = 120 * DAY_LEDGERS;
pub const PERSIST_THRESHOLD: u32 = 30 * DAY_LEDGERS;
pub const RESOLVED_CLAIM_BUMP: u32 = 90 * DAY_LEDGERS;
```

- Every public entry point calls `env.storage().instance().extend_ttl(INSTANCE_THRESHOLD, INSTANCE_BUMP)`.
- Every persistent read or write of a commitment, open claim, provider or template extends that key with `PERSIST_*`.
- A resolved claim is re-written with `RESOLVED_CLAIM_BUMP` and not extended again; history lives in events and the indexer (spec §10.5).
- `ClaimByPayment` is never shortened: duplicate protection must outlive the claim window.
- Bumps must stay below the network's maximum entry TTL. Read it from the network config once and assert in the deploy script.
- The observer keeper calls `extend_commitment_ttl` for every active commitment once a day and `extend_claim_ttl` for every open claim (spec T13).

## 8.4 Event catalogue

Define each event as a struct with the `#[contractevent]` macro so `stellar events` and the generated bindings decode them from the contract spec. If the pinned SDK lacks the macro, fall back to `env.events().publish((symbol_short!("topic"), key), data)` with the same topic and field order.

| Contract | Topic | Indexed key | Data fields |
|---|---|---|---|
| registry | `init` | | admin |
| registry | `adm_prop`, `adm_set` | | address |
| registry | `act_prop` | action_id | kind tag, eta |
| registry | `act_exec`, `act_canc` | action_id | |

| Contract | Topic | Indexed key | Data fields |
|---|---|---|---|
| registry | `prov_reg` | provider_id | admin, home_domain |
| registry | `prov_sig`, `prov_meta` | provider_id | new value |
| registry | `prov_ver`, `prov_unv`, `prov_sus`, `prov_rein` | provider_id | reason (sus only) |
| registry | `tmpl_prop` | template_id | version, activates_at |
| registry | `tmpl_act`, `tmpl_canc`, `tmpl_dep` | template_id | version |
| registry | `obs_add`, `obs_rm` | observer | key (add only) |
| bond | `commit` | commitment_id | provider_id, template_id, version, stake_asset, amount |
| bond | `stake_chg` | commitment_id | delta, new_total |
| bond | `cmt_pause`, `cmt_resum` | commitment_id | at |
| bond | `exit_req`, `exit_fin` | commitment_id | at, amount (fin only) |
| bond | `claim_new` | claim_id | commitment_id, payment_id, breach, penalty, reserved, shortfall, bond, claimant, beneficiary |
| bond | `claim_chal` | claim_id | counter_evidence_hash, attestation_deadline |
| bond | `claim_att` | claim_id | observer, uphold, attestation_hash |
| bond | `claim_res` | claim_id | outcome, amount_paid, bond_refunded |
| bond | `upg_prop`, `upg_exec` | | wasm_hash |
| scoreboard | `bond_set` | | bond |
| scoreboard | `batch` | commitment_id | period, merkle_root, observer, stats |
| scoreboard | `claim_out` | claim_id | provider_id, outcome |

The indexer (§13) treats this table as its contract. A new event or field means a new indexer handler in the same pull request.

### 8.5 Error handling rules

- Every fallible path returns `Result<T, Error>`. No `unwrap`, no `expect`, no `panic!` in contract code. The only tolerated panics are inside SDK host functions (`require_auth`, `ed25519_verify`, token transfer failures).
- Validate before any state write or transfer. A function that fails must leave no partial state; Soroban rolls back on error, but writing checks first keeps reasoning simple.
- Map every `verdict` `None` to `ArithmeticOverflow` or `InvalidEvidence` as noted in §4.5.
- Error codes are append-only. Never renumber.

## 9. Contract testing strategy

Four layers of tests guard the contracts: verdict vectors, per-function unit tests, a full three-contract integration harness, and a property test that runs random action sequences and checks every invariant in §6.9 after each step.

### 9.1 Shared test harness

Create `contracts/bond/src/testutils.rs` (behind `#[cfg(test)]`) that every integration test uses. It deploys all three contracts and a stellar asset contract in one `Env`.

```
pub struct World<'a> {
    pub env: Env,
    pub admin: Address,
    pub usdc: Address,                       // SAC address
    pub usdc_admin: StellarAssetClient<'a>,  // mint
    pub usdc_token: TokenClient<'a>,         // balances, transfers
    pub registry: RegistryClient<'a>,
    pub bond: BondClient<'a>,
    pub scoreboard: ScoreboardClient<'a>,
    pub observer: Address,
    pub provider_admin: Address,
    pub provider_signer: SigningKey,         // ed25519-dalek, for Path B
receipts
    pub client: Address,
```

```
}

impl<'a> World<'a> {
    pub fn new() -> Self;                          // deploy, construct,
set_bond, add observer
    pub fn with_template_901(&self) -> u32;        // propose, advance time
past timelock, activate
    pub fn with_provider(&self) -> BytesN<32>;     // register + verify
    pub fn with_commitment(&self, stake: i128) -> u64;
    pub fn advance_secs(&self, secs: u64);         //
env.ledger().with_mut(|l| { l.timestamp += secs; l.sequence_number += secs as
u32 / 5 })
    pub fn payment(&self, commitment: u64, amount: i128, t0_ms: u64, t1_ms:
Option<u64>) -> PaymentRef;
    pub fn receipt(&self, p: &PaymentRef) -> CompletionReceipt;  // signs
§4.4 bytes
    pub fn assert_invariants(&self);              // checks I1, I2, I7, I8 by
reading storage + token balance
}
```

Register contracts with constructor arguments using `env.register(Contract, (arg1, arg2))`, and the asset with `env.register_stellar_asset_contract_v2(issuer)`. Advance ledger sequence along with timestamp so TTL behaviour is realistic.

## 9.2 Required unit tests

Names are normative; agents should create exactly these so reviewers can find them.

**Registry (`contracts/registry/src/test.rs`)**

- [ ] `constructor_rejects_bad_params` (one case per rule in §5.6)
- [ ] `register_provider_derives_expected_id` (uses `test-vectors/ids.json`)
- [ ] `register_provider_rejects_duplicate`, `_uppercase_domain`, `_too_many_signers`, `_duplicate_signers`
- [ ] `update_signers_requires_provider_admin`
- [ ] `verify_domain_requires_verifier`
- [ ] `propose_template_forces_status_and_eta`
- [ ] `propose_template_rejects_each_invalid_field` (table-driven)
- [ ] `template_version_must_increment`
- [ ] `activate_template_before_eta_fails`, `activate_template_after_eta_succeeds`

- [ ] `deprecated_template_not_returned_by_get_template`
- [ ] `observer_add_and_remove_via_timelock`
- [ ] `provider_admin_cannot_become_observer`
- [ ] `admin_transfer_is_two_step`
- [ ] `upgrade_action_respects_timelock`
- [ ] `auth_matrix_registry` (every row of §8.2 for registry)

### Bond ( `contracts/bond/src/test/*.rs` )

- [ ] `commit_happy_path_moves_tokens_and_emits_event`
- [ ] `commit_rejects_unverified_provider` , `_suspended_provider` , `_inactive_template` , `_asset_mismatch` , `_below_min_stake` , `_below_max_penalty` , `_over_cap`
- [ ] `commit_freezes_exit_cooldown_per_corrected_formula`
- [ ] `add_stake_and_withdraw_excess_respect_min_stake_and_reserved`
- [ ] `pause_resume_coverage_windows` (t0 before, inside, after the pause)
- [ ] `file_claim_path_a_late_completion`
- [ ] `file_claim_path_a_no_completion_after_deadline`
- [ ] `file_claim_rejects_pending_payment` (no t1, before deadline)
- [ ] `file_claim_rejects_on_time_payment`
- [ ] `file_claim_rejects_duplicate_payment_id`
- [ ] `file_claim_rejects_payment_id_mismatch`
- [ ] `file_claim_rejects_out_of_scope_amount`
- [ ] `file_claim_rejects_not_covered_t0` (before effective_from, during pause, after exit request)
- [ ] `file_claim_rejects_after_claim_window`
- [ ] `file_claim_rejects_unknown_observer`
- [ ] `file_claim_requires_exactly_one_path`
- [ ] `file_claim_path_b_valid_receipt` and `_wrong_signer` , `_tampered_amount`
- [ ] `file_claim_partial_reserve_records_shortfall`
- [ ] `resolve_before_challenge_deadline_fails`
- [ ] `resolve_unchallenged_pays_beneficiary_and_refunds_bond`
- [ ] `challenge_after_deadline_fails` , `challenge_requires_provider_admin`
- [ ] `attest_uphold_then_resolve_pays`

- [ ] `attest_reject_then_resolve_splits_bond`
- [ ] `attest_twice_fails`, `attest_after_window_fails`
- [ ] `challenged_without_quorum_expires_after_window`
- [ ] `resolve_twice_fails`
- [ ] `exit_blocked_by_open_claim`, `exit_blocked_by_cooldown`, `exit_returns_full_stake`
- [ ] `claim_against_exiting_commitment_still_pays`
- [ ] `scoreboard_receives_outcome_on_every_resolution`
- [ ] `auth_matrix_bond`
- [ ] `ttl_extended_on_touch` (assert `env.storage().persistent().get_ttl(..)` after calls)

**Scoreboard (`contracts/scoreboard/src/test.rs`)**

- [ ] `set_bond_only_once`
- [ ] `anchor_batch_requires_active_observer`
- [ ] `anchor_batch_rejects_misaligned_period`, `_future_period`, `_duplicate`, `_inconsistent_counts`
- [ ] `record_claim_outcome_rejects_non_bond_caller`
- [ ] `lifetime_accumulates_batches_and_outcomes`
- [ ] `periods_list_capped_at_90`

## 9.3 Property test

`contracts/bond/tests/properties.rs` uses `proptest` to generate sequences of 1 to 60 actions from this alphabet, applied to a `World` with two providers, two commitments and one observer:

`Commit`, `AddStake(amount)`, `Withdraw(amount)`, `Pause`, `Resume`, `RequestExit`, `FinalizeExit`, `FileClaimA(amount, lateness_ms, has_t1)`, `FileClaimB(..)`, `Challenge`, `Attest(uphold)`, `Resolve`, `AdvanceTime(secs)`.

Rules:

- Actions are allowed to fail. The test records the error and continues.
- After every action, successful or not, call `world.assert_invariants()`.

- At the end, advance time past every deadline, resolve every open claim, finalize every exit, and assert the contract's token balance is exactly zero and that the sum of all balances across providers, clients and observers equals what was minted (conservation check).
- Run with `PROPTEST_CASES=1000` in CI and 20,000 locally before mainnet.

### 9.4 Fuzzing (after the weekend)

`cargo fuzz` target over `file_claim` and `resolve` with arbitrary `PaymentRef` and timestamps, asserting no panic other than auth failures and that invariants hold. Tracked as VX-913.

### 9.5 Testnet smoke tests

`scripts/smoke.ts` runs after every deploy: reads `params`, reads template 901, calls `available_stake` on the seeded commitment and simulates (does not submit) a `file_claim` that must fail with `NotInBreach`. A failing smoke test blocks the demo.

## 10. Testnet deployment

Two scripts take a clean checkout to a seeded testnet deployment: `deploy.ts` builds and deploys the three contracts and writes their IDs, and `seed.ts` creates test USDC, the observer, template 901, a verified provider and a funded commitment.

### 10.1 Accounts

Create and fund these with the CLI (Friendbot funds XLM for fees):

```
for k in deployer usdc-issuer observer provider-admin client verifier; do
  stellar keys generate $k --network testnet --fund
done
```

| Alias | Role | Notes |
|---|---|---|
| `deployer` | Registry, bond and scoreboard admin on testnet | Single key is fine on testnet only |
| `usdc-issuer` | Issues the test USDC asset | Lets the team mint freely; do not depend on an external faucet for the demo |

| Alias | Role | Notes |
|---|---|---|
| `observer` | Reference observer | Its secret goes in `OBSERVER_SECRET` |
| `provider-admin` | Mock anchor's provider admin | Holds test USDC for stake |
| `client` | Sending side, claim beneficiary | Holds test USDC to pay the mock anchor |
| `verifier` | Calls `verify_domain` | Set with `propose(SetVerifier)`, or leave as deployer for the weekend |

Using Circle's testnet USDC instead is possible (it lets the demo use the real asset code and issuer), but minting is limited by their faucet. Keep it as a config switch, `USDC_SOURCE=self|circle`, default `self`.

## 10.2 Test USDC

`seed.ts` does this with `@stellar/stellar-sdk`; the CLI equivalents are shown for debugging.

1. Asset `USDC:<usdc-issuer public key>`.
2. Deploy its Stellar Asset Contract: `stellar contract asset deploy --asset USDC:$ISSUER --source deployer --network testnet`. Record the contract ID as `usdc` in the deployments file. Re-running is safe: if it already exists, read the ID with `stellar contract id asset --asset USDC:$ISSUER --network testnet`.
3. Add a trustline from `provider-admin`, `client` and `observer` (the observer pays claim bonds) to the asset.
4. Pay 100,000 USDC to `provider-admin`, 10,000 to `client`, 1,000 to `observer` from the issuer.

## 10.3 Deploy order (D15)

```
stellar contract build   # optimised wasm by default in CLI 27.x

REG=$(stellar contract deploy --network testnet --source deployer \
  --wasm target/wasm32v1-none/release/vaeltrix_registry.wasm --alias
vaeltrix_registry \
  -- --admin deployer --params "$(cat config/params.testnet.json)")

SCB=$(stellar contract deploy --network testnet --source deployer \
  --wasm target/wasm32v1-none/release/vaeltrix_scoreboard.wasm --alias
vaeltrix_scoreboard \
  -- --admin deployer --registry $REG)

BOND=$(stellar contract deploy --network testnet --source deployer \
  --wasm target/wasm32v1-none/release/vaeltrix_bond.wasm --alias
vaeltrix_bond \
  -- --admin deployer --registry $REG --scoreboard $SCB)

stellar contract invoke --network testnet --source deployer --id $SCB --
set_bond --bond $BOND
```

`config/params.testnet.json` holds the §5.7 testnet column, with i128 values as strings and the USDC SAC ID filled in by `deploy.ts` before construction (so test USDC must exist first: run step 10.2 items 1 and 2 before this).

`deploy.ts` wraps exactly these steps, refuses to overwrite an existing `deployments/testnet.json` unless `--fresh` is passed, and writes:

```
{
  "network": "testnet",
  "rpcUrl": "https://soroban-testnet.stellar.org",
  "deployedAt": "2026-10-17T00:00:00Z",
  "gitSha": "<commit>",
  "contracts": {
    "registry": "C...",
    "bond": "C...",
    "scoreboard": "C...",
    "usdc": "C..."
  },
  "wasmHashes": { "registry": "...", "bond": "...", "scoreboard": "..." },
  "accounts": { "admin": "G...", "observer": "G...", "usdcIssuer": "G..." },
  "startLedger": 0
}
```

`startLedger` is the ledger of the first deploy transaction. The indexer starts there.

## 10.4 Seed order

`seed.ts` is idempotent: each step reads state first and skips if already done.

1. Test USDC, trustlines, balances (§10.2).
2. `registry.propose(AddObserver(observer, observer_ed25519_key))`, wait `timelock_secs + 5`, `registry.execute(id)`.
3. `registry.propose_template(901)` with the §6.10 values, wait, `activate_template(901, 1)`.
4. Optionally propose templates 1 to 3 as well so the explorer has realistic data. They need no activation for the demo.
5. `registry.register_provider(provider-admin, "anchor.mock.vaeltrix.xyz", [mock anchor signer pubkey], "https://anchor.mock.vaeltrix.xyz/vaeltrix.json")`.
6. `registry.verify_domain(provider_id)` from the verifier. For the weekend, verification is manual; VX-207 automates the toml check.
7. `bond.commit(provider_id, 901, usdc, 1_000_0000000 /* 1,000 USDC */, 0, 0)`.
8. Write `providerId` and `commitmentId` into the deployments file under `seed`.

With §5.7 values, the waits add about three minutes total. The script logs each wait so nobody kills it thinking it hung.

### 10.5 Bindings

```
for c in registry bond scoreboard; do
  stellar contract bindings typescript --network testnet \
    --contract-id $(jq -r .contracts.$c deployments/testnet.json) \
    --output-dir packages/bindings/$c --overwrite
done
pnpm -r --filter "./packages/bindings/*" build
```

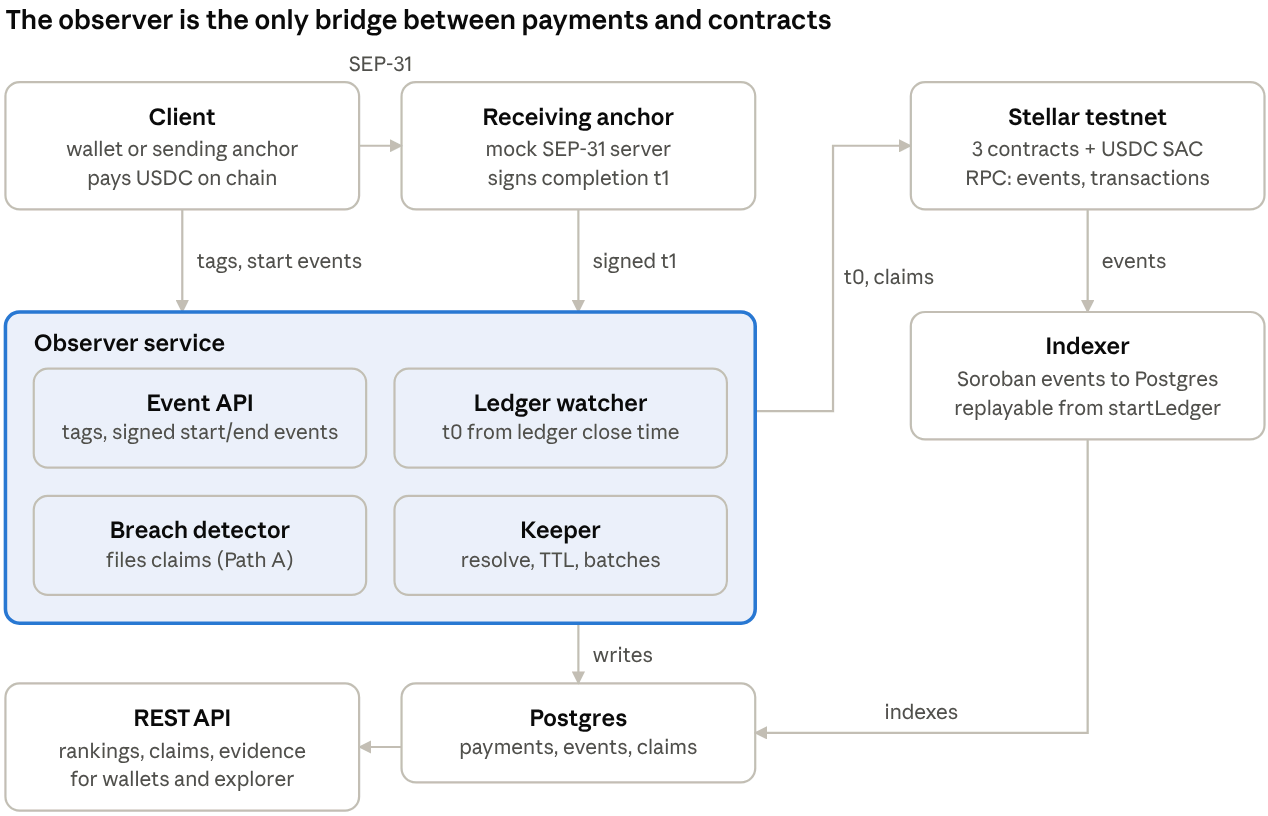
Bindings are generated, never hand-edited. Commit them so services build without network access, and regenerate after every contract interface change.

### 10.6 Redeploying

On testnet, prefer a fresh deploy over an upgrade while interfaces are changing: `pnpm deploy:testnet --fresh && pnpm seed:testnet && pnpm bindings`. Reset the indexer database after a fresh deploy ( `pnpm db:reset` ). Once interfaces stabilise, exercise the upgrade path at least once ( `propose_upgrade` , wait, `execute_upgrade` ) so mainnet is not the first time it runs.

## 11. Backend architecture and Postgres schema

The off-chain side is four Node services sharing one Postgres database: the observer turns payment reality into claims, the indexer mirrors contract events, the API serves reads, and a mock anchor stands in for a real provider until pilots sign.




V0 backend architecture · 4 services, 1 database, 3 contracts

The observer writes payment timelines and signed events; the indexer writes only what the chain emitted. The API reads both and never writes to the chain.

## 11.1 Service responsibilities

| Service | Package | Writes to chain | Writes to Postgres | Exposes |
|---|---|---|---|---|
| Observer | `services/observer` | `file_claim`, `resolve`, `attest`, `anchor_batch`, `extend_*_ttl` | `payments`, `signed_events`, `evidence_bundles`, `observer_jobs`, `attestations` | `POST /v1/payments`, `POST /v1/events`, `POST /v1/claims/delegate`, `GET /healthz` on port 4001 |
| Indexer | `services/indexer` | nothing | `chain_events`, `providers`, `templates`, `commitments`, `claims`, `claim_votes`, `batches`, `cursors` | `GET /healthz` on port 4002 |

| Service | Package | Writes to chain | Writes to Postgres | Exposes |
|---|---|---|---|---|
| API | `services/api` | nothing | nothing (read-only role) | `GET /v1/*` on port 4000 |
| Mock anchor | `services/mock-anchor` | nothing (it receives USDC) | its own tables in schema `mock` | SEP-31-shaped `GET /info`, `POST /transactions`, `GET /transactions/:id` on port 4100 |

The observer and indexer both see contract state. The rule: the observer may read `commitments` and `claims` written by the indexer, but whenever a decision moves money (filing, resolving) it re-reads state from the contract by simulation first. The indexer is a cache, never the authority.

## 11.2 Postgres schema

Migrations live in `db/migrations/NNNN_name.sql`, applied in order, never edited after merge. Amounts are `numeric(39,0)` (fits i128). Timestamps from the chain are `bigint` milliseconds plus a generated `timestamptz` column for humans.

```
-- 0001_core.sql
create table cursors (
  name text primary key,                -- 'indexer:events',
'observer:ledger'
  ledger bigint not null,
  paging_token text,
  updated_at timestamptz not null default now()
);

create table chain_events (
  id text primary key,                  -- RPC event id, unique and ordered
  ledger bigint not null,
  ledger_closed_at timestamptz not null,
  tx_hash text not null,
  contract text not null,               -- 'registry' | 'bond' |
'scoreboard'
  topic text not null,
  key text,
  data jsonb not null,
  processed boolean not null default false
);
```

```
create index on chain_events (contract, topic);
create index on chain_events (processed) where not processed;

create table providers (
  provider_id text primary key,          -- hex
  admin text not null,
  home_domain text not null,
  metadata_uri text,
  signers text[] not null,               -- hex ed25519 pubkeys
  verified boolean not null default false,
  status text not null,                  -- active | suspended | exited
  registered_at timestamptz not null
);

create table templates (
  template_id int not null,
  version int not null,
  body jsonb not null,                   -- full SlaTemplate as JSON (spec
6.2 shape)
  status text not null,
  activates_at timestamptz,
  primary key (template_id, version)
);

create table commitments (
  commitment_id bigint primary key,
  provider_id text not null references providers,
  template_id int not null,
  template_version int not null,
  stake_asset text not null,
  stake_total numeric(39,0) not null,
  stake_reserved numeric(39,0) not null default 0,
  status text not null,
  effective_from timestamptz not null,
  exit_requested_at timestamptz,
  updated_ledger bigint not null
);

create table payments (
  payment_id text primary key,           -- hex, D4
  sep31_tx_id text not null,
  provider_id text not null references providers,
  commitment_id bigint references commitments,   -- null = accepted without
```

```
coverage
  client_account text,                  -- G... of the sender side, if
registered
  beneficiary text,                     -- address that receives penalties
  amount numeric(39,0) not null,
  asset text not null,
  sender_country text, receiver_country text, payout_rail text,
  stellar_tx_hash text,
  t0_ms bigint,                         -- ledger close of settlement
  t1_ms bigint,                         -- signed completion
  pause_ms bigint not null default 0,
  status text not null,                 -- tagged | settled | completed |
breached | carved_out | out_of_scope | claimed | resolved
  verdict text,                         -- pending | on_time | late |
no_completion
  latency_ms bigint,
  evidence_hash text,
  claim_id bigint,
  delegated boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create index on payments (status);
create index on payments (commitment_id, t0_ms);
create unique index on payments (provider_id, sep31_tx_id);

create table signed_events (
  id bigserial primary key,
  payment_id text not null references payments,
  event text not null,                  -- receiver_credit_confirmed,
compliance_hold_declared, ...
  ts_ms bigint not null,
  signer text not null,                 -- hex pubkey
  body jsonb not null,                  -- full signed event as received
  body_hash text not null,              -- JCS sha256 without sig
  verified boolean not null,
  received_at timestamptz not null default now()
);

create table evidence_bundles (
  evidence_hash text primary key,
  payment_id text not null references payments,
  body jsonb not null,
```

```
  created_at timestamptz not null default now()
);

create table claims (
  claim_id bigint primary key,
  commitment_id bigint not null,
  payment_id text not null,
  breach text not null,
  claimant text not null, beneficiary text not null,
  penalty numeric(39,0) not null, reserved numeric(39,0) not null,
  shortfall numeric(39,0) not null, bond numeric(39,0) not null,
  status text not null,                  -- filed | challenged | paid |
rejected | expired
  outcome text,
  filed_at timestamptz not null,
  challenge_deadline timestamptz not null,
  attestation_deadline timestamptz,
  counter_evidence_hash text,
  amount_paid numeric(39,0),
  resolved_at timestamptz,
  filed_tx text not null,
  resolved_tx text
);
create index on claims (status, challenge_deadline);

create table claim_votes (
  claim_id bigint not null references claims,
  observer text not null,
  uphold boolean not null,
  attestation_hash text not null,
  primary key (claim_id, observer)
);

create table attestations (               -- observer's own published log
(spec 13.3)
  attestation_hash text primary key,
  claim_id bigint,
  body jsonb not null,
  created_at timestamptz not null default now()
);

create table batches (
  commitment_id bigint not null,
```

```
  period bigint not null,                -- day index, D14
  observer text not null,
  merkle_root text not null,
  stats jsonb not null,
  records_uri text,                      -- where the full record set is
downloadable
  anchored_tx text,
  primary key (commitment_id, period)
);

create table delegations (
  client_account text not null,
  provider_id text,                      -- null = all providers
  beneficiary text not null,
  created_at timestamptz not null default now(),
  revoked_at timestamptz,
  primary key (client_account, beneficiary)
);

create table observer_jobs (
  id bigserial primary key,
  kind text not null,                    -- file_claim | resolve | attest |
anchor_batch | extend_ttl
  ref text not null,                     -- payment_id, claim_id, ...
  status text not null,                  -- queued | running | submitted |
confirmed | failed
  attempts int not null default 0,
  last_error text,
  tx_hash text,
  run_after timestamptz not null default now(),
  unique (kind, ref)
);
```

The `unique (kind, ref)` constraint on jobs is what stops the observer from filing two claims for one payment after a restart. The contract would reject the second (`DuplicateClaim`), but it would still cost a bond transfer attempt and a fee.

## 11.3 Shared library: @vaeltrix/core

Every service imports these and nothing re-implements them:

| Module | Exports |
|---|---|
| `ids` | `providerId(homeDomain, adminG)`, `paymentId(sep31TxId, providerIdHex)` |
| `jcs` | `canonicalize(obj)`, `jcsHash(obj)` |
| `receipt` | `receiptBytes(fields, networkPassphrase)`, `signReceipt`, `verifyReceipt` (§4.4) |
| `events` | `signEvent(event, keypair)`, `verifyEvent(event, pubkeys)` (§12.4) |
| `verdict` | TypeScript port of every §4.5 function, using `bigint` throughout |
| `merkle` | `merkleRoot(records)`, `merkleProof(records, id)` (§7.4) |
| `amounts` | `toStroops('12.5')`, `fromStroops(125000000n)` with 7 decimals |
| `config` | Loads `deployments/<network>.json` and env vars with zod validation |

All money and time values in TypeScript are `bigint`. A lint rule bans `Number(` on any variable named `*amount*`, `*_ms`, `*stake*`.

## 12. Observer service

The observer is one Node process with an HTTP server and seven background loops; it decides breach from ledger time and signed events, files claims under Path A, and keeps the chain moving by resolving claims and extending TTLs.

### 12.1 Process layout

```
services/observer/src/
├── index.ts                # boots config, db, chain client, http, loops
├── config.ts               # zod-validated env + deployments file
├── chain/
│   ├── client.ts           # bindings clients + keypair signer
│   ├── submit.ts           # single serialized submission queue (§12.10)
```

```
|   └── ledger.ts            # latestLedgerCloseMs(), getTx(hash), horizon
fallback
├── http/
|   ├── auth.ts              # signed-request auth (§12.3)
|   ├── payments.ts          # POST /v1/payments
|   ├── events.ts            # POST /v1/events
|   ├── delegate.ts          # POST /v1/claims/delegate
|   └── evidence.ts          # POST /v1/counter-evidence, GET
/v1/evidence/:hash
├── loops/
|   ├── settlement.ts        # resolves t0 for payments with a tx hash
|   ├── sep31Poller.ts       # stretch: polls provider GET /transactions/:id
|   ├── breachDetector.ts    # verdicts, evidence bundles, enqueues
file_claim
|   ├── jobRunner.ts         # executes observer_jobs
|   ├── keeper.ts            # resolve + extend_ttl
|   ├── attester.ts          # reacts to claim_chal
|   └── batchPublisher.ts    # daily anchor_batch
└── evidence/
    ├── bundle.ts            # build + hash evidence bundle
    └── attestation.ts       # build + hash attestation document
```

Each loop is a `setInterval`-style async function guarded so two ticks never overlap, with its interval in config. One process is enough for V0; every loop is idempotent so a crash and restart is safe.

## 12.2 Payment lifecycle inside the observer

| From | Trigger | To | Notes |
|---|---|---|---|
| (none) | `POST /v1/payments` accepted with coverage | `tagged` | `commitment_id` set |
| (none) | `POST /v1/payments` out of scope or uncovered | `out_of_scope` | Stored for the scoreboard, never claimable |
| `tagged` | t0 resolved from ledger (§12.5) | `settled` | `t0_ms`, `stellar_tx_hash` set |
| `settled` | valid `receiver_credit_confirmed` | `completed` | `t1_ms` set, verdict computed at once |

| From | Trigger | To | Notes |
|---|---|---|---|
| `completed` or `settled` | verdict `late` or `no_completion` | `breached` | evidence bundle written |
| `breached` | claim job confirmed on chain | `claimed` | `claim_id` set |
| `claimed` | `claim_res` indexed | `resolved` | |
| any before `breached` | `invalid_beneficiary` event with refund tx | `carved_out` | Leaves SLA scope |

A payment `completed` on time stays `completed` with verdict `on_time`; it only feeds the daily batch.

## 12.3 Write API authentication

Spec §18.4 calls for SEP-10 and SEP-45. For the weekend, use signed requests, which are simpler and just as strong for machine clients; SEP-10 becomes VX-617.

- Header `X-Vaeltrix-Key`: hex ed25519 public key (a provider signer, or a client key registered via delegation).
- Header `X-Vaeltrix-Timestamp`: Unix ms; rejected if more than 30 s from the observer clock.
- Header `X-Vaeltrix-Signature`: base64 ed25519 signature over `METHOD\nPATH\nTIMESTAMP\nsha256hex(body)`.
- Provider endpoints accept only keys in that provider's on-chain `signers` (read from the indexer table, refreshed on `prov_sig`).
- A replay cache keyed by signature keeps 60 s of seen requests.

## 12.4 Endpoints

**`POST /v1/payments`** (provider signer). Tags a payment. Body:

```
{
  "sep31_tx_id": "82fhs729f63dh0v4",
  "commitment_id": 1,
  "amount": "1000000000",
  "asset": "USDC:G...ISSUER",
  "sender_country": "US", "receiver_country": "NG", "payout_rail": "BANK",
  "client_account": "G...CLIENT",
  "beneficiary": "G...CLIENT"
}
```

Checks, in order: commitment exists and belongs to this provider; scope (asset, countries, rail, amount band) against the template; commitment `is_covered(now)` by simulation; `available_stake >= max_penalty` by simulation (D5); optional volume ceiling for the current period. Returns `201 { "vaeltrix_payment_id": "<hex>", "covered": true }`, or `200 { "covered": false, "reason": "out_of_scope" }`. Idempotent on `(provider_id, sep31_tx_id)`: a repeat returns the original response.

**`POST /v1/events`** (provider or client key). Submits one signed event:

```
{
  "v": 1,
  "network": "testnet",
  "event": "receiver_credit_confirmed",
  "payment_id": "b3f1...9a",
  "sep31_tx_id": "82fhs729f63dh0v4",
  "provider_id": "7c2e...41",
  "commitment_id": 1,
  "ts": "2026-10-17T14:03:22.418Z",
  "rail_ref": "NIP-000013261011140322",
  "signer": "<hex pubkey>",
  "sig": "ed25519:<base64>"
}
```

`sig` is ed25519 over the UTF-8 bytes of `JCS(event without the sig field)`. This adds `network` and `signer` to the spec §7.7 format. Accepted event kinds and their effect:

| Event | Signed by | Extra fields | Effect |
|---|---|---|---|
| `settlement_submitted` | client | `stellar_tx_hash` | Gives the observer the tx to resolve t0 from. Ledger is still the authority. |

| Event | Signed by | Extra fields | Effect |
|---|---|---|---|
| `receiver_credit_confirmed` | provider | `rail_ref` | Sets t1. Rejected if `ts` is more than 30 s ahead of observer clock, or earlier than t0. |
| `compliance_hold_declared` | provider | `hold_code` | Opens a pause span. Only before the deadline. |
| `compliance_hold_released` | provider | | Closes the span. Unreleased holds close at `max_hold_ms`. |
| `sender_info_requested` / `sender_info_supplied` | provider / client | | Pause span for `sender_info_incomplete`. |
| `invalid_beneficiary` | provider | `refund_tx_hash` | Leaves scope only after the refund tx is found on ledger. |

The first valid `receiver_credit_confirmed` wins. A second one with a different `ts` is stored, flagged, and attached to any later evidence bundle as a conflict (useful in disputes, spec T1).

**`POST /v1/claims/delegate`** (client key). Body `{ "client_account", "beneficiary", "provider_id"?: hex }`. Authorises the observer to file and pay the bond on the client's behalf. For the weekend demo the client delegates once during seeding.

**`POST /v1/counter-evidence`** (provider signer). Body is any JSON; the observer stores it, returns its `jcsHash`, which the provider then passes to `bond.challenge`.

**`GET /v1/evidence/:hash`** (public). Returns an evidence bundle, attestation document or counter-evidence by hash.

## 12.5 Resolving t0

Given a `stellar_tx_hash` (from `settlement_submitted` or the SEP-31 poller):

1. `rpc.getTransaction(hash)`. If not found (outside RPC retention), fall back to Horizon `GET /transactions/:hash` and `/transactions/:hash/operations`.
2. Require `SUCCESS`, the network passphrase, and exactly one payment operation to one of the provider's receiving accounts for the tagged `asset` and `amount`. Receiving accounts come from the provider metadata JSON at `metadata_uri` (`{"receiving_accounts": ["G..."]}`), cached for 10 minutes.
3. If the SEP-31 flow uses a memo, require the memo to match what the provider returned for that transaction.

4. `t0_ms = ledger close time (seconds) * 1000`. Store, set `settled`.

Any mismatch leaves the payment `tagged` and logs a warning with the reason. A payment that never settles never breaches; the SLA clock has not started.

## 12.6 Breach detector

Every 5 seconds:

1. `now_ms = latestLedgerCloseMs()` from RPC, never the server clock.
2. Select payments in `settled` or `completed` with `verdict` null or `pending`.
3. Apply carve-outs with `core.verdict.applyCarveOuts` to get `pause_ms`.
4. `verdict = core.verdict.evaluate(t0, t1, now, target, grace, pause)`.
5. On `late`, or on `no_completion` once `now_ms > deadline_ms + 10_000` (10 s safety margin so the contract, which sees a slightly later ledger, agrees), build the evidence bundle, store it, set `breached`.
6. If the payment is delegated (or the client is the observer's own demo client), enqueue `observer_jobs(kind='file_claim', ref=payment_id)`.

Evidence bundle (hashed with JCS, stored in `evidence_bundles`, served at `/v1/evidence/:hash`):

```
{
  "v": 1,
  "payment_id": "...", "sep31_tx_id": "...", "provider_id": "...",
"commitment_id": 1,
  "template": { "id": 901, "version": 1 },
  "amount": "1000000000", "asset": "USDC:G...",
  "settlement": { "stellar_tx_hash": "...", "ledger": 123456, "close_time":
"...", "t0_ms": 1760710000000 },
  "completion": { "event": { ...signed event as received... }, "t1_ms":
1760710150000 },
  "carve_outs": [ { "kind": "compliance_hold_declared", "from_ms": 0,
"to_ms": 0, "events": [] } ],
  "pause_ms": 0,
  "verdict": { "kind": "late", "latency_ms": 150000, "deadline_ms":
1760710130000 },
  "conflicts": [],
  "observer": "G...OBS",
  "computed_at_ledger": 123500
}
```

### 12.7 Claim filer (job runner, kind `file_claim`)

1. Re-read the payment. Skip if `claim_id` set.
2. Simulate `bond.claim_by_payment(payment_id)`. If `Some(id)`, record it, mark `claimed`, done.
3. Build `PaymentRef` and call `bond.file_claim(claimant=observer, beneficiary, payment, observer=Some(observer), receipt=None)` through the submission queue.
4. Map contract errors: `DuplicateClaim` go to step 2; `ClaimWindowClosed` sets payment `resolved` with note `missed_window` and alerts (this is an observer failure); `NotInBreach` retries once after 15 s then fails loudly; `NotCovered`, `OutOfScope` mark `out_of_scope` and alert (tagging should have prevented it); anything else retries with exponential backoff up to 5 attempts.
5. On success store `claim_id` from the return value and set `claimed`.

The observer pays the claim bond. Alert when its USDC balance falls below 10 bonds' worth or its XLM below 50.

### 12.8 Keeper

Every 15 seconds:

- Claims `filed` with `challenge_deadline < ledger now`: enqueue `resolve`.
- Claims `challenged` where votes reach quorum, or `attestation_deadline < ledger now`: enqueue `resolve`.
- Before submitting, simulate; on `ChallengeWindowOpen` or `QuorumNotReached` requeue with `run_after` at the deadline.

Once a day: `extend_commitment_ttl` for every non-exited commitment and `extend_claim_ttl` for every open claim.

### 12.9 Attester

On each indexed `claim_chal`:

1. Load the claim's evidence bundle and the counter-evidence by hash. Missing counter-evidence counts as no new evidence.
2. Re-run the verdict over the union: a provider-signed `receiver_credit_confirmed` with valid signature and `ts` inside target plus grace (that the original bundle lacked) flips to reject; a signed carve-out that was declared before the deadline is applied and the verdict recomputed; anything else upholds.

3. Build the attestation document (spec §13.3 plus `network`), store it in `attestations`, compute its `jcsHash`.
4. Call `bond.attest(observer, claim_id, uphold, attestation_hash)`.

The verdict function used here MUST be the shared one. A reviewer must be able to rerun it from published data and get the same answer (spec §13.4 determinism).

## 12.10 Transaction submission

- One queue per signing account; submissions are strictly serial so sequence numbers never collide.
- Use the generated bindings: `client.method(args)` returns an assembled transaction; sign with the observer `Keypair`, `signAndSend()`, then poll `getTransaction` until `SUCCESS` or `FAILED`, timeout 60 s.
- On `txBadSeq` reload the account and retry once. On `txInsufficientFee` retry with a higher inclusion fee.
- Persist `tx_hash` on the job before polling so a restart can resume polling rather than resubmit.
- Decode contract errors to the §4.3 names in logs (`Error(Contract, #7)` becomes `DuplicateClaim`).

## 12.11 Batch publisher (stretch)

At 00:10 UTC, for each active commitment and the previous UTC day: select payments with `t0_ms` in the period; compute `PeriodStats` (nearest-rank percentiles over on-time and late latencies, carved-out excluded from on-time rate); compute the §7.4 Merkle root; write the record set to `GET /v1/batches/:commitment/:period/records`; build and store the attestation document; call `scoreboard.anchor_batch`. Empty periods are anchored too, with a zero root, so gaps are visible.

## 12.12 Observability

- Logs: `pino`, JSON, one line per state change with `payment_id` or `claim_id`.
- `GET /healthz`: db reachable, RPC reachable, latest ledger age under 30 s, observer XLM and USDC balances.
- `GET /metrics` (Prometheus): payments by status, verdict counts, claims filed, jobs failed, loop durations.

- Alerts (log level `error` plus a webhook URL if configured): job failed permanently, missed claim window, quorum unreachable, low balance, ledger stale.

# 13. Indexer and REST API

The indexer replays contract events into Postgres projections from `startLedger`, and the API serves those projections plus the observer's evidence; both are fully rebuildable, so neither ever needs a backup.

## 13.1 Indexer loop

1. Read cursor `indexer:events` (ledger plus paging token). On first run start at `deployments.startLedger`.
2. `rpc.getEvents({ startLedger | cursor, filters: [{ type: 'contract', contractIds: [registry, bond, scoreboard] }], limit: 200 })`.
3. For each event, in order, inside one database transaction: insert into `chain_events` (`on conflict (id) do nothing`), apply the projection handler for its topic, then advance the cursor.
4. Sleep 2 s when the page is empty, otherwise loop immediately.

Decode topics and data with `scValToNative`. Stellar has deterministic finality, so there are no reorgs to handle; ordering by event id is final.

RPC keeps a limited event history. If the cursor falls outside retention (indexer down too long), the handler logs an error and runs **reconcile**: read every provider, template, commitment and open claim directly from contract storage through view calls and rewrite the projections. Reconcile is also exposed as `pnpm indexer:reconcile`.

## 13.2 Projection handlers

| Topic | Handler effect |
|---|---|
| `prov_reg` | Insert provider (signers fetched by simulating `get_provider`) |
| `prov_sig`, `prov_meta`, `prov_ver`, `prov_unv`, `prov_sus`, `prov_rein` | Update provider fields |
| `tmpl_prop`, `tmpl_act`, `tmpl_canc`, `tmpl_dep` | Upsert template row (body fetched with `get_template_version`) |

| Topic | Handler effect |
|---|---|
| `obs_add`, `obs_rm` | Upsert into an `observers` view table |
| `commit` | Insert commitment |
| `stake_chg`, `cmt_pause`, `cmt_resum`, `exit_req`, `exit_fin` | Update commitment |
| `claim_new` | Insert claim; update `payments.claim_id` and status `claimed` if the payment is known |
| `claim_chal` | Status `challenged`, attestation deadline |
| `claim_att` | Insert vote, bump counts |
| `claim_res` | Status from outcome, `amount_paid`, `resolved_at`; reserved stake released in commitment row; payment status `resolved` |
| `batch` | Insert batch |
| `claim_out` | No-op (lifetime stats are read live from the scoreboard view) |

Every handler is idempotent. A unit test replays a recorded event fixture twice and asserts identical tables.

## 13.3 REST API conventions

- Base path `/v1`, JSON, `snake_case` keys.
- Amounts are decimal strings in stroops plus a `_display` sibling with 7-decimal formatting (`"stake_total": "10000000000", "stake_total_display": "1000.0000000"`).
- Timestamps are ISO 8601 UTC strings; raw ms values keep the `_ms` suffix.
- List endpoints take `limit` (default 50, max 200) and an opaque `cursor`, and return `{ "items": [], "next_cursor": null }`.
- Errors: `{ "error": { "code": "not_found", "message": "..." } }` with matching HTTP status.
- CORS open for GET. Responses cached 5 s in memory.
- An OpenAPI document is generated from Fastify route schemas and served at `/v1/openapi.json`.

## 13.4 Endpoints

| Method and path | Returns | Weekend |
|---|---|---|
| `GET /v1/providers` | Providers with `verified`, `status`, headline score, total stake | must |
| `GET /v1/providers/:id` | Profile, commitments, lifetime stats (from scoreboard view), claim counts | must |
| `GET /v1/commitments/:id` | Commitment with `available_stake`, `max_penalty`, coverage ratio | must |
| `GET /v1/commitments/:id/stats?period=30d` | `PeriodStats` series from batches | stretch |
| `GET /v1/templates` and `/v1/templates/:id` | Template bodies in the spec §6.2 JSON shape | must |
| `GET /v1/claims?status=&provider=&commitment=` | Claim list | must |
| `GET /v1/claims/:id` | Claim, votes, evidence hash, attestation hashes, tx links | must |
| `GET /v1/payments/:payment_id` | Payment timeline: tagged, t0, t1, carve-outs, verdict, claim | must |
| `GET /v1/evidence/:hash` | Proxied from the observer | must |
| `GET /v1/corridors/:corridor/rankings?template=1` | Providers ranked by score for e.g. `US-NG` | stretch |
| `GET /v1/batches/:commitment/:period` | Root, stats, records link | stretch |
| `GET /v1/network` | Contract IDs, network, latest indexed ledger, lag in seconds | must |

Example `GET /v1/claims/12`:

```
{
  "claim_id": 12,
  "status": "paid",
  "outcome": "upheld",
  "breach": "late_completion",
  "commitment_id": 1,
  "provider_id": "7c2e...41",
  "payment_id": "b3f1...9a",
  "penalty": "50000000", "penalty_display": "5.0000000",
  "reserved": "50000000", "shortfall": "0",
  "amount_paid": "50000000",
  "bond": "10000000",
  "claimant": "G...OBS", "beneficiary": "G...CLIENT",
  "filed_at": "2026-10-17T14:05:40Z",
  "challenge_deadline": "2026-10-17T14:08:40Z",
  "resolved_at": "2026-10-17T14:08:55Z",
  "evidence_hash": "e91c...07",
  "votes": [],
  "transactions": { "filed": "<hash>", "resolved": "<hash>" },
  "explorer_links": { "filed":
"https://stellar.expert/explorer/testnet/tx/<hash>" }
}
```

## 13.5 Derived metrics

Computed in the API, never stored on chain (spec §14.3):

- `on_time_rate_30d` = on-time over (payments minus carved-out) across batches in the last 30 days. Before batches exist, compute live from the observer's `payments` table and flag `source: "observer_live"`.
- `carve_out_rate` = carved-out over payments.
- `coverage_ratio` (V0 definition) = `available_stake / (max_penalty * in_flight)`, where `in_flight` counts tagged payments whose claim deadline has not passed. With zero in flight, report 1.5 (the cap).
- `score = 100 * on_time_rate_30d * (1 - min(carve_out_rate / 0.2, 0.5)) * min(coverage_ratio / 1.5, 1)`.
- Fewer than 100 tagged payments in 30 days returns `score: null, score_status: "insufficient_data"`.

The formula is versioned: responses carry `score_version: "0.1"`, so wallets know when it changes.

# 14. TypeScript SDK and mock anchor

`@vaeltrix/sdk` is a thin, typed layer over the generated bindings and the observer HTTP API, and the mock anchor is the first program built on it, so the SDK is proven by the demo before any real anchor sees it.

## 14.1 SDK construction

```
import { Vaeltrix } from "@vaeltrix/sdk";

const vx = new Vaeltrix({
  network: "testnet",              // loads contract IDs from the bundled
deployments file
  rpcUrl: process.env.STELLAR_RPC_URL,
  observerUrl: "http://localhost:4001",
  apiUrl: "http://localhost:4000",
  signer: Keypair.fromSecret(secret),      // signs Soroban transactions
  eventKey: ed25519SeedHex,                // optional: signs Vaeltrix events
and API requests
});
```

## 14.2 SDK surface

Every method returns typed objects with `bigint` amounts. Write methods return `{ txHash, result }`. Read methods simulate and never submit.

| Module | Method | Backed by |
|---|---|---|
| `providers` | `register({ homeDomain, signers, metadataUri })` | registry `register_provider` |
| | `get(providerId)`, `idFor(homeDomain, admin)` | registry views, `core.ids` (asserts both agree) |
| | `updateSigners(providerId, signers)` | registry |
| `templates` | `get(id)`, `list()` | registry views |

| Module | Method | Backed by |
|---|---|---|
| `commitments` | `create({ providerId, templateId, stakeAsset, amount, maxVolumePerPeriod?, periodSecs? })` | bond `commit` |
| | `addStake`, `withdrawExcess`, `pause`, `resume`, `requestExit`, `finalizeExit` | bond |
| | `get(id)`, `availableStake(id)`, `isCovered(id, t0Ms)` | bond views |
| `payments` | `tag({ commitmentId, sep31TxId, amount, asset, corridor, payoutRail, clientAccount?, beneficiary? })` | observer `POST /v1/payments` |
| | `get(paymentId)` | API |
| `events` | `signCompletion({ paymentId, sep31TxId, providerId, commitmentId, railRef, ts })` | builds, signs and posts `receiver_credit_confirmed` |
| | `signSettlement({ paymentId, stellarTxHash })` | client-side `settlement_submitted` |
| | `declareHold`, `releaseHold`, `declareInvalidBeneficiary` | provider carve-out events |
| `claims` | `file({ payment, beneficiary, receipt })` | bond `file_claim` Path B (stretch) |
| | `delegate({ beneficiary, providerId? })` | observer `POST /v1/claims/delegate` |
| | `challenge(claimId, counterEvidence)` | posts counter-evidence, then bond `challenge` |
| | `resolve(claimId)`, `get(claimId)` | bond, API |
| `receipts` | `sign(payment)`, `verify(payment, receipt, pubkey)` | `core.receipt` |

| Module | Method | Backed by |
|---|---|---|
| `scoreboard` | `get({ providerId, corridor?, templateId? })`, `rankings(corridor, templateId)` | API |

Errors are thrown as `VaeltrixError` with a `code` that matches the §4.3 names (`"DuplicateClaim"`), never raw XDR.

## 14.3 Mock anchor behaviour

A small Fastify server that behaves like a SEP-31 receiving anchor closely enough for the demo, signs completions with a key registered as the provider's signer, and can be told how to misbehave.

| Endpoint | Behaviour |
|---|---|
| `GET /info` | SEP-31-shaped info for USDC plus the `vaeltrix` extension (spec §17.2) listing the seeded commitment |
| `POST /transactions` | Body: `amount`, `asset_code`, `asset_issuer`, `vaeltrix_commitment_id`, optional `fields`. Creates an id, calls `vx.payments.tag(..)`, returns `{ id, stellar_account_id, stellar_memo_type: "text", stellar_memo, vaeltrix_payment_id }`. Rejects with 400 `customer_info_needed` style errors only if asked to. |
| `GET /transactions/:id` | SEP-31 status (`pending_sender`, `pending_receiver`, `completed`, `error`), `stellar_transaction_id`, `completed_at`, `amount_in`, `vaeltrix_payment_id` |
| `GET /.well-known/stellar.toml` | Serves the `[VAELTRIX]` block (spec §17.1) so VX-207 can verify it |
| `GET /vaeltrix.json` | Provider metadata with `receiving_accounts` (§12.5) |

Internals:

1. Watches its receiving account for USDC payments (Horizon payments stream, matched by memo).
2. On match: status `pending_receiver`, records the tx hash, and schedules the payout.
3. After the configured delay: status `completed`, `completed_at = now`, then `vx.events.signCompletion(..)` to the observer.

Behaviour is set per transaction with header `X-Mock-Behaviour` on `POST /transactions`, defaulting to `MOCK_ANCHOR_DEFAULT_BEHAVIOUR`:

| Value | Effect | Used by |
|---|---|---|
| `on_time` | Completes after 30 s | happy path check |
| `late` | Completes after `MOCK_ANCHOR_PAYOUT_DELAY_MS` (150 s for template 901, deadline 130 s) | `demo:breach` |
| `never` | Never completes | no-completion breach |
| `hold` | Declares a compliance hold at 20 s, releases at 80 s, completes at 140 s | carve-out check (on time after pause) |
| `backdate` | Completes late but signs a `ts` 60 s earlier than real | dispute tests, spec T1 |
| `invalid_beneficiary` | Refunds on chain and sends `invalid_beneficiary` | out-of-scope check |

The mock anchor is test infrastructure. It MUST print a startup banner saying so and refuse to start with `STELLAR_NETWORK=public`.

# 15. End-to-end demo script

`pnpm demo:breach` proves the whole protocol in about seven minutes on testnet: a late payment becomes a paid penalty with no human step, and the script asserts the client's balance rises by exactly the penalty.

## 15.1 Preconditions

- `deployments/testnet.json` exists and `pnpm seed:testnet` has run (template 901 active, provider verified, commitment funded, client delegated).

- `pnpm dev` is running: Postgres, observer, indexer, API, mock anchor.
- `GET /v1/network` reports indexer lag under 10 s.

The script checks all three and exits with a clear message if any fails.

## 15.2 Steps and expected timing

| # | Step | Actor | Expected result | Approx time |
|---|---|---|---|---|
| 1 | Read client USDC balance `B0` and commitment `available_stake` `S0` | script | numbers printed | 0:00 |
| 2 | `POST /transactions` to mock anchor, 100 USDC, `X-Mock-Behaviour: late` | client | `vaeltrix_payment_id`, memo, receiving account | 0:02 |
| 3 | Pay 100 USDC on chain with the memo | client | tx hash `H` | 0:08 |
| 4 | `vx.events.signSettlement({ paymentId, stellarTxHash: H })` | client | 202 from observer | 0:09 |
| 5 | Poll `GET /v1/payments/:id` until `settled` | script | `t0_ms` equals ledger close of `H` | 0:15 |
| 6 | Wait | mock anchor | completes at t0 + 150 s, posts signed t1 | 2:40 |
| 7 | Poll until `breached` then `claimed` | observer | verdict `late`, latency about 150,000 ms, claim id `C` | 2:50 |
| 8 | Poll `GET /v1/claims/C` until `paid` | keeper | challenge window (180 s) passes, `resolve` submitted | 6:00 |
| 9 | Read client balance `B1` and `available_stake` `S1` | script | `B1 - B0 == 5 USDC` and `S0 - S1 == 5 USDC` | 6:05 |

| # | Step | Actor | Expected result | Approx time |
|---|---|---|---|---|
| 10 | Print summary with stellar.expert links for steps 3, 7 and 8 | script | | 6:05 |

With template 901 the penalty for 100 USDC is `min(100 * 500 / 10_000, 100) = 5 USDC`.

The observer files on the client's behalf and pays the bond, so the bond refund goes to the observer, not the client. The client's balance change is exactly the penalty.

## 15.3 Variants

| Script | Mock behaviour | Expected outcome |
|---|---|---|
| `pnpm demo:breach` | `late` | Claim paid, 5 USDC to client |
| `pnpm demo:ontime` | `on_time` | No claim; payment verdict `on_time` |
| `pnpm demo:nocompletion` | `never` | Claim filed at deadline plus 10 s, breach `no_completion`, paid |
| `pnpm demo:hold` | `hold` | No claim; `pause_ms = 60000`, verdict `on_time` |
| `pnpm demo:challenge` | `late` | Provider challenges within 180 s with counter-evidence that adds nothing; observer attests uphold; claim paid after quorum |
| `pnpm demo:reject` | `backdate` then a provider-signed honest t1 inside target posted as counter-evidence | Observer attests reject; reserve released; bond split (D13) |

## 15.4 Output format

The script prints one line per step with elapsed time and a final block that a person can paste into an SCF update:

```
Vaeltrix testnet demo (breach) · 2026-10-18T15:42:10Z
provider    7c2e...41  commitment 1  template 901 v1
payment     b3f1...9a  100.0000000 USDC  t0 15:35:12Z  t1 15:37:42Z  latency
150.0s  target 120s + 10s grace
claim       #3  filed 15:37:55Z  paid 15:40:58Z  penalty 5.0000000 USDC
client      +5.0000000 USDC   stake 1000.0000000 -> 995.0000000 USDC
links       settle https://stellar.expert/explorer/testnet/tx/...
            claim  https://stellar.expert/explorer/testnet/tx/...
            paid   https://stellar.expert/explorer/testnet/tx/...
```

# 16. GitHub issues backlog

Nine epics and 63 issues cover V0; 50 carry the `weekend` label and form the minimum path to the testnet demo, and every issue below is a complete, copy-ready body.

## 16.1 Labels

| Label | Meaning |
|---|---|
| `epic:<name>` | One of: foundations, registry, bond, scoreboard, deploy, observer, indexer-api, sdk-mock, demo-docs |
| `area:contracts`, `area:backend`, `area:tooling` | Where the code lives |
| `weekend` | Must land by Oct 18 for the demo |
| `stretch` | Weekend if time allows |
| `post-weekend` | V0 but after the demo |
| `size:S`, `size:M`, `size:L` | Under 2 h, 2 to 5 h, over 5 h for one engineer or one agent session |
| `blocked` | A dependency is not merged |

## 16.2 Issue body format

Every issue follows this shape so humans and agents can parse it:

```
### VX-305 · Title in imperative form
**Labels:** epic:bond, area:contracts, weekend, size:L
**Depends on:** VX-302, VX-303
**Spec:** §6.6, §2 D2 D5
**Context:** why this exists, one to three sentences.
**Tasks:** numbered steps.
**Acceptance criteria:** checklist; every item is testable.
**Out of scope:** what not to do here.
```

To create them in bulk, export this document to Markdown, then split on lines starting with `### VX-` and call `gh issue create --title "<heading>" --body-file <chunk> --label <labels>` per chunk. A 30-line script in `scripts/issues.ts` (VX-904) does this and skips titles that already exist.

### 16.3 Critical path

VX-101, VX-103, VX-104 then in parallel (VX-201 to VX-205) and (VX-301 to VX-307) then VX-401, VX-502, VX-503, VX-504 then VX-601 to VX-608 with VX-802 then VX-901. Everything else hangs off that line.

## Epic 1: Foundations (VX-1xx)

### VX-101 · Scaffold the monorepo

**Labels:** epic:foundations, area:tooling, weekend, size:S **Depends on:** none **Spec:** §3.1, §3.3, §3.4 **Context:** every other issue assumes the layout in §3.1. **Tasks:**

1. Create the directory tree in §3.1 with placeholder crates and packages that compile.
2. Root `Cargo.toml` per §3.3, `rust-toolchain.toml` with stable plus `wasm32v1-none`.
3. `pnpm-workspace.yaml`, root `package.json` with every script in §3.4 (scripts may be stubs).
4. `.env.example` with every variable in §3.5, `.gitignore` (target, node_modules, .env, test_snapshots).

**Acceptance criteria:**

- [ ] `cargo build --workspace` and `pnpm -r build` succeed on a clean clone.
- [ ] `stellar contract build` produces three wasm files.

**Out of scope:** any contract logic.

## VX-102 · Add CI workflow

**Labels:** epic:foundations, area:tooling, weekend, size:S **Depends on:** VX-101 **Spec:** §3.6 **Tasks:** one workflow, jobs `rust`, `ts`, `vectors`, cache cargo and pnpm, upload wasm artifacts. **Acceptance criteria:**

- [ ] All three jobs run on pull requests and on `main`.
- [ ] Clippy runs with `-D warnings`.
- [ ] Branch protection on `main` requires the three jobs.

## VX-103 · Implement vaeltrix-types

**Labels:** epic:foundations, area:contracts, weekend, size:M **Depends on:** VX-101 **Spec:** §4.1, §4.2, §4.3 **Context:** shared by all contracts. Must be frozen early (§1.4 step 1). **Tasks:**

1. Every enum and struct in §4.1 and §4.2 exactly as written.
2. `Error` enum in §4.3 with the exact codes.
3. `RegistryApi` and `ScoreboardApi` client traits.
4. Constants: `MAX_PAUSE_MS`, TTL constants from §8.3.
5. Helper `fn provider_id(env, home_domain, admin) -> BytesN<32>` and `fn payment_id(env, sep31_tx_id, provider_id) -> BytesN<32>` per §2 D4.

**Acceptance criteria:**

- [ ] Crate compiles for `wasm32v1-none` and host.
- [ ] ID helpers match every case in `test-vectors/ids.json`.

**Out of scope:** contract logic.

## VX-104 · Implement vaeltrix-verdict

**Labels:** epic:foundations, area:contracts, weekend, size:M **Depends on:** VX-101 **Spec:** §4.5 **Tasks:** implement every function in §4.5 including `apply_carve_outs`, using only checked arithmetic, no panics, no Soroban dependency. **Acceptance criteria:**

- [ ] `#![no_std]`, zero dependencies.
- [ ] Passes every case in `test-vectors/verdict.json` (VX-105).
- [ ] Unit test proves `penalty` rounds down.

### VX-105 · Write the shared test vectors

**Labels:** epic:foundations, area:tooling, weekend, size:M **Depends on:** VX-101 **Spec:** §4.6, §4.4 **Tasks:** author `verdict.json` (at least 40 cases covering the §4.6 checklist), `ids.json` (at least 5), `receipts.json` (at least 3 with a fixed test seed, hex bytes and signature). Generate expected values with a throwaway script and hand-check five of them. **Acceptance criteria:**

- [ ] Every checklist item in §4.6 has at least one case.
- [ ] Files validate against a JSON schema committed next to them.

### VX-106 · Build @vaeltrix/core

**Labels:** epic:foundations, area:backend, weekend, size:L **Depends on:** VX-105 **Spec:** §11.3, §4.4, §4.5, §7.4, §12.4 **Tasks:**

1. `jcs` (RFC 8785; use a vetted library such as `canonicalize`), `ids`, `receipt`, `events`, `verdict` (bigint port), `merkle`, `amounts`, `config`.
2. Vitest suite that loads all three vector files.

**Acceptance criteria:**

- [ ] Every vector passes in TypeScript.
- [ ] `receiptBytes` output is byte-identical to the Rust test (shared vector).
- [ ] No `number` type used for amounts or ms values anywhere in the package.

### VX-107 · Docker Compose and migrations runner

**Labels:** epic:foundations, area:backend, weekend, size:S **Depends on:** VX-101 **Spec:** §11.2 **Tasks:** `docker-compose.yml` with Postgres 16; `db/migrations/0001_core.sql` exactly as §11.2; `pnpm db:migrate` and `pnpm db:reset`. **Acceptance criteria:**

- [ ] `pnpm db:reset && pnpm db:migrate` succeeds twice in a row.

## Epic 2: Registry (VX-2xx)

### VX-201 · Registry skeleton, constructor and timelocked actions

**Labels:** epic:registry, area:contracts, weekend, size:M **Depends on:** VX-103 **Spec:** §5.1, §5.2, §5.6 **Tasks:** `DataKey`, `ActionKind`, `PendingAction`; `__constructor`; `propose`, `execute`, `cancel`; two-step admin; params validation; `Upgrade` action calling `update_current_contract_wasm`. **Acceptance criteria:**

- [ ] Tests `constructor_rejects_bad_params`, `admin_transfer_is_two_step`, `upgrade_action_respects_timelock` pass.
- [ ] `execute` before eta returns `TimelockActive`.

### VX-202 · Provider registration and management

**Labels:** epic:registry, area:contracts, weekend, size:M **Depends on:** VX-201 **Spec:** §5.3, §2 D4 **Tasks:** `register_provider`, `update_signers`, `update_metadata`, `verify_domain`, `unverify_domain`, `suspend_provider`, `reinstate_provider`, views. **Acceptance criteria:**

- [ ] All provider tests in §9.2 pass.
- [ ] Derived IDs match `ids.json`.

### VX-203 · Template lifecycle

**Labels:** epic:registry, area:contracts, weekend, size:M **Depends on:** VX-201 **Spec:** §5.4 **Tasks:** `propose_template` with every validation rule, `activate_template`, `cancel_template`, `deprecate_template`, `get_template`, `get_template_version`, `template_ids`. **Acceptance criteria:**

- [ ] Table-driven test covers every validation rule in §5.4 step 2.
- [ ] Caller-supplied `status` and `activates_at` are ignored.

### VX-204 · Observer set management

**Labels:** epic:registry, area:contracts, weekend, size:S **Depends on:** VX-201, VX-202 **Spec:** §5.5 **Acceptance criteria:**

- [ ] Add and remove work only through propose and execute.
- [ ] Provider admin cannot be added as observer.
- [ ] `is_active_observer` and `active_observer_count` correct after removal.

### VX-205 · Registry events and TTL

**Labels:** epic:registry, area:contracts, weekend, size:S **Depends on:** VX-202, VX-203, VX-204 **Spec:** §8.3, §8.4 **Acceptance criteria:**

- [ ] Every registry row of the §8.4 table is emitted, asserted in tests via `env.events().all()`.
- [ ] Instance and persistent TTL extended on touch (test `ttl_extended_on_touch`).

### VX-206 · Registry auth matrix test

**Labels:** epic:registry, area:contracts, weekend, size:S **Depends on:** VX-205 **Spec:** §8.2
**Acceptance criteria:**

- [ ] For each registry row, right signer passes and wrong signer fails using `mock_auths`, not `mock_all_auths`.

### VX-207 · Domain verifier script

**Labels:** epic:registry, area:tooling, stretch, size:S **Depends on:** VX-202, VX-504 **Spec:** §5.3, spec §17.1 **Tasks:** `scripts/verify-domain.ts <provider_id>`: fetch `https://<home_domain>/.well-known/stellar.toml`, parse `[VAELTRIX]`, compare `PROVIDER_ID` and `REGISTRY_CONTRACT`, call `verify_domain` or `unverify_domain`.
**Acceptance criteria:**

- [ ] Verifies the mock anchor's toml; refuses on a mismatched ID with a clear message.

## Epic 3: Bond (VX-3xx)

### VX-301 · Bond skeleton, constructor and upgrade timelock

**Labels:** epic:bond, area:contracts, weekend, size:S **Depends on:** VX-103 **Spec:** §6.1, §6.2 **Tasks:** `DataKey`, constructor storing admin, registry, scoreboard; `propose_upgrade`, `execute_upgrade`, `cancel_upgrade` reading `timelock_secs` from registry params.
**Acceptance criteria:**

- [ ] Constructor stores addresses; views return them.
- [ ] Upgrade before eta fails with `TimelockActive`.
- [ ] No function lets the admin move stake or change a claim.

### VX-302 · Commit, add stake, withdraw excess

**Labels:** epic:bond, area:contracts, weekend, size:L **Depends on:** VX-301, VX-104, VX-202, VX-203 **Spec:** §6.4, §6.5, §2 D8 **Tasks:**

1. `commit` with all 11 steps in §6.5, including corrected `exit_cooldown_secs` and frozen `min_stake`.
2. `add_stake`, `withdraw_excess` per the §6.5 table.
3. Views `commitment`, `available_stake`, `max_penalty`, `commitments_by_provider`.

**Acceptance criteria:**

- [ ] Every `commit_*` test in §9.2 passes.
- [ ] `add_stake_and_withdraw_excess_respect_min_stake_and_reserved` passes.
- [ ] Token balances asserted before and after, not just storage.

### VX-303 · Coverage rule, pause and resume

**Labels:** epic:bond, area:contracts, weekend, size:M **Depends on:** VX-302 **Spec:** §6.3, §2 D6 **Acceptance criteria:**

- [ ] `is_covered` private fn and public view share one implementation.
- [ ] `pause_resume_coverage_windows` covers t0 before, inside and after the pause, and after a second pause.

### VX-304 · Exit flow

**Labels:** epic:bond, area:contracts, weekend, size:S **Depends on:** VX-302 **Spec:** §6.5 table, I3 **Acceptance criteria:**

- [ ] `exit_blocked_by_open_claim`, `exit_blocked_by_cooldown`, `exit_returns_full_stake` pass.
- [ ] `claim_against_exiting_commitment_still_pays` passes (needs VX-307; mark that test ignored until then).

### VX-305 · file_claim, Path A

**Labels:** epic:bond, area:contracts, weekend, size:L **Depends on:** VX-303, VX-204 **Spec:** §6.6, §2 D2 D4 D5 D7 **Tasks:** implement steps 1 to 17 in §6.6 for the observer path; Path B arguments accepted but return `InvalidEvidence` until VX-308. **Acceptance criteria:**

- [ ] Every `file_claim_*` test in §9.2 except the Path B ones passes.
- [ ] Checks run in the documented order (a test files an invalid claim that fails two checks and asserts the earlier error).
- [ ] Partial reservation sets `shortfall` correctly.

### VX-306 · Challenge and attest

**Labels:** epic:bond, area:contracts, weekend, size:M **Depends on:** VX-305 **Spec:** §6.7 **Acceptance criteria:**

- [ ] `challenge_after_deadline_fails`, `challenge_requires_provider_admin`, `attest_twice_fails`, `attest_after_window_fails` pass.

- [ ] Votes stored in `ClaimVotes` with observer and direction.

### VX-307 · Resolve, payouts, bond split, scoreboard call

**Labels:** epic:bond, area:contracts, weekend, size:L **Depends on:** VX-306, VX-401 **Spec:** §6.7 resolve table, §2 D10 D11 D13 **Acceptance criteria:**

- [ ] Every row of the resolve table has a test.
- [ ] Storage updated before transfers; scoreboard called after transfers; event last.
- [ ] `scoreboard_receives_outcome_on_every_resolution` passes.
- [ ] Bond split leaves no stroop unaccounted for.

### VX-308 · file_claim, Path B (provider-signed receipts)

**Labels:** epic:bond, area:contracts, stretch, size:M **Depends on:** VX-305, VX-105 **Spec:** §4.4, §6.6 step 11 **Acceptance criteria:**

- [ ] Contract builds the 131-byte message identical to `receipts.json`.
- [ ] `file_claim_path_b_valid_receipt`, `_wrong_signer`, `_tampered_amount` pass.
- [ ] Path B with no t1 fails with `NotInBreach`.

### VX-309 · Bond TTL and keeper functions

**Labels:** epic:bond, area:contracts, weekend, size:S **Depends on:** VX-307 **Spec:** §8.3 **Acceptance criteria:**

- [ ] `extend_commitment_ttl` and `extend_claim_ttl` are permissionless.
- [ ] Resolved claims rewritten with the 90-day bump; `ClaimByPayment` never shortened.

### VX-310 · Three-contract test harness

**Labels:** epic:bond, area:contracts, weekend, size:M **Depends on:** VX-201, VX-301, VX-401 **Spec:** §9.1 **Tasks:** `World` struct and helpers exactly as §9.1, including `assert_invariants` for I1, I2, I7, I8. **Acceptance criteria:**

- [ ] One smoke test builds a `World`, commits, and asserts invariants.
- [ ] Harness is reusable from `tests/properties.rs`.

### VX-311 · Bond auth matrix test

**Labels:** epic:bond, area:contracts, weekend, size:S **Depends on:** VX-307 **Spec:** §8.2 **Acceptance criteria:**

- [ ] Every bond row of §8.2 tested with right and wrong signer via `mock_auths`.

### VX-312 · Property test over random action sequences

**Labels:** epic:bond, area:contracts, weekend, size:L **Depends on:** VX-310, VX-307 **Spec:** §9.3, §6.9 **Acceptance criteria:**

- [ ] 1,000 cases in CI, all invariants checked after every action, conservation check at the end.
- [ ] A deliberately broken build (reserve not released on reject) is caught by the test; describe how you checked this in the pull request.

### VX-318 · Cache params in bond

**Labels:** epic:bond, area:contracts, post-weekend, size:S **Depends on:** VX-307 **Spec:** §8.1 **Context:** only if simulation shows `file_claim` close to resource limits. **Acceptance criteria:**

- [ ] `sync_params()` copies registry params into bond instance storage; claims read the cache.

## Epic 4: Scoreboard (VX-4xx)

### VX-401 · Scoreboard contract

**Labels:** epic:scoreboard, area:contracts, weekend, size:M **Depends on:** VX-103, VX-204 **Spec:** §7.1, §7.2, §2 D14 D15 **Tasks:** constructor, `set_bond`, `anchor_batch`, `record_claim_outcome`, views, events, TTL. **Acceptance criteria:**

- [ ] Every scoreboard test in §9.2 passes.
- [ ] `record_claim_outcome` from any address other than bond fails.

### VX-402 · Merkle root reference implementation in Rust

**Labels:** epic:scoreboard, area:contracts, post-weekend, size:S **Depends on:** VX-106 **Spec:** §7.4 **Context:** contracts do not compute roots, but a Rust test crate that matches `@vaeltrix/core` gives auditors a second implementation. **Acceptance criteria:**

- [ ] Same roots as TypeScript for a shared `merkle.json` vector file.

## Epic 5: Deploy (VX-5xx)

### VX-501 · Testnet accounts and test USDC

**Labels:** epic:deploy, area:tooling, weekend, size:S **Depends on:** VX-101 **Spec:** §10.1, §10.2 **Tasks:** generate and fund the six accounts; deploy the USDC SAC; trustlines; initial balances. Code lives in `scripts/lib/usdc.ts` and is called from `seed.ts`; asset deploy also callable standalone because `deploy.ts` needs the SAC ID first. **Acceptance criteria:**

- [ ] Re-running is a no-op.
- [ ] `USDC_SOURCE=circle` switch documented, even if untested.

### VX-502 · deploy.ts

**Labels:** epic:deploy, area:tooling, weekend, size:M **Depends on:** VX-201, VX-301, VX-401, VX-501 **Spec:** §10.3, §2 D15 **Acceptance criteria:**

- [ ] Deploys in the D15 order, calls `set_bond`, writes the deployments JSON with `startLedger` and wasm hashes.
- [ ] Refuses to overwrite without `--fresh`.
- [ ] Asserts TTL constants are below the network maximum.

### VX-503 · seed.ts

**Labels:** epic:deploy, area:tooling, weekend, size:M **Depends on:** VX-502 **Spec:** §10.4, §6.10 **Acceptance criteria:**

- [ ] All eight steps run idempotently and log every timelock wait.
- [ ] Template 901 active, provider verified, commitment funded with 1,000 test USDC, client delegated with the observer (calls VX-609 endpoint if the observer is up, otherwise prints the command to run later).

### VX-504 · Bindings generation

**Labels:** epic:deploy, area:tooling, weekend, size:S **Depends on:** VX-502 **Spec:** §10.5 **Acceptance criteria:**

- [ ] `pnpm bindings` regenerates and builds the three packages; they are committed.

- [ ] A type-check in `packages/sdk` fails if a binding is stale (CI compares generated output with committed output).

### VX-505 · Testnet smoke test

**Labels:** epic:deploy, area:tooling, weekend, size:S **Depends on:** VX-503, VX-504 **Spec:** §9.5 **Acceptance criteria:**

- [ ] `pnpm smoke:testnet` passes on the seeded deployment and fails clearly against an unseeded one.

### VX-506 · Params config files

**Labels:** epic:deploy, area:tooling, weekend, size:S **Depends on:** VX-103 **Spec:** §5.7 **Acceptance criteria:**

- [ ] `config/params.testnet.json` and `config/params.mainnet.example.json` match §5.7 and pass the §5.6 rules in a unit test that runs the Rust validator through a tiny test binary or mirrors it in TypeScript.

## Epic 6: Observer (VX-6xx)

### VX-601 · Observer scaffold, config and submission queue

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-106, VX-107, VX-504 **Spec:** §12.1, §12.10 **Acceptance criteria:**

- [ ] Service boots, validates config with zod, connects to Postgres and RPC, serves `GET /healthz`.
- [ ] `submit.ts` serializes transactions, persists `tx_hash` before polling, decodes contract errors to §4.3 names.
- [ ] Loop runner prevents overlapping ticks (unit test).

### VX-602 · Signed-request authentication

**Labels:** epic:observer, area:backend, weekend, size:S **Depends on:** VX-601 **Spec:** §12.3 **Acceptance criteria:**

- [ ] Rejects bad signature, stale timestamp, replay, and keys not in the provider's signer set.
- [ ] Signer set refreshes from the indexer table without restart.

## VX-603 · POST /v1/payments (tagging)

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-602 **Spec:** §12.4, §2 D5 **Acceptance criteria:**

- [ ] All checks in order; out-of-scope payments stored with `covered: false`.
- [ ] Idempotent on `(provider_id, sep31_tx_id)`.
- [ ] Returns 409 when available stake is below max penalty.

## VX-604 · POST /v1/events

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-602 **Spec:** §12.4 events table **Acceptance criteria:**

- [ ] Verifies JCS signature, signer authority per event kind, clock skew of 30 s.
- [ ] First valid completion wins; later conflicting completion stored and flagged.
- [ ] Unit tests for each event kind in the table.

## VX-605 · t0 resolution from the ledger

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-603, VX-604 **Spec:** §12.5 **Acceptance criteria:**

- [ ] Uses RPC `getTransaction`, falls back to Horizon.
- [ ] Rejects a tx paying the wrong account, asset, amount or memo, with a logged reason.
- [ ] `t0_ms` equals ledger close time in seconds times 1,000.

## VX-606 · Breach detector and evidence bundles

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-605 **Spec:** §12.6 **Acceptance criteria:**

- [ ] Uses ledger time, never wall clock.
- [ ] 10 s safety margin on no-completion.
- [ ] Evidence bundle hash is reproducible from `GET /v1/evidence/:hash` output.
- [ ] Enqueues exactly one `file_claim` job per breached, delegated payment.

## VX-607 · Claim filer job

**Labels:** epic:observer, area:backend, weekend, size:M **Depends on:** VX-606 **Spec:** §12.7 **Acceptance criteria:**

- [ ] Every error mapping in §12.7 step 4 has a test with a mocked contract response.
- [ ] Restarting the observer mid-filing never produces a second on-chain claim attempt for the same payment.

### VX-608 · Keeper: resolve and TTL

**Labels:** epic:observer, area:backend, weekend, size:S **Depends on:** VX-607, VX-702 **Spec:** §12.8 **Acceptance criteria:**

- [ ] Resolves unchallenged claims within 30 s after the challenge deadline on testnet.
- [ ] Requeues on `ChallengeWindowOpen` and `QuorumNotReached` without counting as failure.

### VX-609 · Delegation endpoint

**Labels:** epic:observer, area:backend, weekend, size:S **Depends on:** VX-602 **Spec:** §12.4 **Acceptance criteria:**

- [ ] Client can delegate and revoke; breach detector honours it.

### VX-610 · Attester

**Labels:** epic:observer, area:backend, stretch, size:M **Depends on:** VX-608, VX-306 **Spec:** §12.9 **Acceptance criteria:**

- [ ] On `claim_chal`, produces an attestation document, stores it, calls `attest` within 60 s.
- [ ] Verdict rules in §12.9 step 2 each covered by a test.

### VX-611 · Batch publisher

**Labels:** epic:observer, area:backend, stretch, size:M **Depends on:** VX-401, VX-106 **Spec:** §12.11, §7.4 **Acceptance criteria:**

- [ ] Anchors one batch per commitment per day, empty days included.
- [ ] Records downloadable; recomputed root matches the on-chain root.

### VX-612 · SEP-31 poller

**Labels:** epic:observer, area:backend, stretch, size:S **Depends on:** VX-605 **Spec:** §12.5 **Acceptance criteria:**

- [ ] Polls `GET /transactions/:id` for tagged payments without a tx hash, picks up `stellar_transaction_id`, and cross-checks `completed_at` against signed t1 (mismatch beyond grace is flagged).

### VX-613 · Observer observability and alerts

**Labels:** epic:observer, area:backend, weekend, size:S **Depends on:** VX-601 **Spec:** §12.12 **Acceptance criteria:**

- [ ] `/healthz` and `/metrics` as specified; low-balance and missed-window alerts fire in a test.

### VX-617 · SEP-10 and SEP-45 authentication

**Labels:** epic:observer, area:backend, post-weekend, size:L **Depends on:** VX-602 **Spec:** spec §17.3, §18.4 **Acceptance criteria:**

- [ ] Observer issues SEP-10 challenges for classic accounts and SEP-45 for contract accounts; JWT accepted on write endpoints alongside signed requests.

## Epic 7: Indexer and API (VX-7xx)

### VX-701 · Indexer loop and cursor

**Labels:** epic:indexer-api, area:backend, weekend, size:M **Depends on:** VX-107, VX-504 **Spec:** §13.1 **Acceptance criteria:**

- [ ] Starts at `startLedger`, pages through `getEvents`, stores `chain_events`, advances cursor in the same transaction.
- [ ] Survives restart without duplicates or gaps (test with a recorded fixture).

### VX-702 · Projection handlers

**Labels:** epic:indexer-api, area:backend, weekend, size:M **Depends on:** VX-701 **Spec:** §13.2, §8.4 **Acceptance criteria:**

- [ ] One handler per topic in §8.4; replaying a fixture twice yields identical tables.
- [ ] Unknown topics are stored and logged, never crash the loop.

### VX-703 · Reconcile from contract state

**Labels:** epic:indexer-api, area:backend, stretch, size:M **Depends on:** VX-702 **Spec:** §13.1
**Acceptance criteria:**

- [ ] `pnpm indexer:reconcile` rebuilds projections from view calls and matches the event-built tables on the demo deployment.

### VX-704 · API scaffold and conventions

**Labels:** epic:indexer-api, area:backend, weekend, size:S **Depends on:** VX-107 **Spec:** §13.3
**Acceptance criteria:**

- [ ] Fastify with route schemas, error shape, pagination helper, amount formatting helper, OpenAPI at `/v1/openapi.json`.
- [ ] Read-only database role used.

### VX-705 · Weekend API endpoints

**Labels:** epic:indexer-api, area:backend, weekend, size:M **Depends on:** VX-702, VX-704
**Spec:** §13.4 rows marked must **Acceptance criteria:**

- [ ] Every must endpoint returns the documented shape; example response for `GET /v1/claims/:id` matches §13.4 field for field.
- [ ] `GET /v1/network` reports indexer lag.

### VX-706 · Derived metrics and rankings

**Labels:** epic:indexer-api, area:backend, stretch, size:M **Depends on:** VX-705 **Spec:** §13.5
**Acceptance criteria:**

- [ ] Score formula unit-tested against hand-computed cases including insufficient data.
- [ ] `GET /v1/corridors/:corridor/rankings` sorted by score, nulls last.

## Epic 8: SDK and mock anchor (VX-8xx)

### VX-801 · @vaeltrix/sdk

**Labels:** epic:sdk-mock, area:backend, weekend, size:L **Depends on:** VX-106, VX-504 **Spec:** §14.1, §14.2 **Tasks:** implement every row of §14.2 except Path B claims (stretch, after VX-308) and `scoreboard.rankings` (after VX-706). **Acceptance criteria:**

- [ ] Integration test against testnet: register a throwaway provider, read it back, `idFor` matches.
- [ ] Errors surface as `VaeltrixError` with §4.3 codes.

### VX-802 · Mock SEP-31 anchor

**Labels:** epic:sdk-mock, area:backend, weekend, size:L **Depends on:** VX-801, VX-603, VX-604 **Spec:** §14.3 **Acceptance criteria:**

- [ ] All endpoints in the §14.3 table.
- [ ] Behaviours `on_time`, `late`, `never` work end to end; `hold`, `backdate`, `invalid_beneficiary` are stretch.
- [ ] Refuses to start on public network.

### VX-803 · Mock anchor stellar.toml and provider metadata

**Labels:** epic:sdk-mock, area:backend, weekend, size:S **Depends on:** VX-802 **Spec:** §10.4 step 5, §12.5, spec §17.1 **Tasks:** serve `/.well-known/stellar.toml` with `[VAELTRIX]` and `/vaeltrix.json` with `receiving_accounts`. For the weekend, the observer may read metadata from `http://localhost:4100` via a config override, since the mock domain is not public. **Acceptance criteria:**

- [ ] Observer t0 resolution finds the receiving account through metadata, not hard-coded config.

## Epic 9: Demo and docs (VX-9xx)

### VX-901 · demo:breach script

**Labels:** epic:demo-docs, area:tooling, weekend, size:M **Depends on:** VX-503, VX-608, VX-705, VX-802 **Spec:** §15.1, §15.2, §15.4 **Acceptance criteria:**

- [ ] Runs unattended from preconditions to summary.
- [ ] Asserts `B1 - B0 == penalty` and `S0 - S1 == penalty` exactly, exits non-zero otherwise.
- [ ] Output matches the §15.4 format.

### VX-902 · Demo variants

**Labels:** epic:demo-docs, area:tooling, stretch, size:M **Depends on:** VX-901, VX-610 **Spec:** §15.3 **Acceptance criteria:**

- [ ] `demo:ontime`, `demo:nocompletion`, `demo:challenge` pass; `demo:hold` and `demo:reject` after their mock behaviours land.

### VX-903 · README and quickstart

**Labels:** epic:demo-docs, area:tooling, weekend, size:S **Depends on:** VX-901 **Acceptance criteria:**

- [ ] A teammate who has not touched the repo reruns the demo from a clean clone in under 15 minutes following only the README.
- [ ] README links this guide (exported to `docs/implementation-guide.md`) and the technical spec.

### VX-904 · Issue import script and guide export

**Labels:** epic:demo-docs, area:tooling, stretch, size:S **Depends on:** VX-101 **Spec:** §16.2 **Acceptance criteria:**

- [ ] `scripts/issues.ts` creates every `VX-` issue with labels via `gh`, skipping existing titles; `--dry-run` prints them.

### VX-913 · Fuzz claim and resolve

**Labels:** epic:demo-docs, area:contracts, post-weekend, size:M **Depends on:** VX-312 **Spec:** §9.4 **Acceptance criteria:**

- [ ] `cargo fuzz` target runs for 30 minutes without an unexpected panic or invariant failure.

## 17. Agent operating rules and prompts

An AI agent working on Vaeltrix takes one `VX-` issue at a time, reads only the sections that issue cites plus §2 to §4, and stops to ask rather than invent when this guide is silent.

## 17.1 Rules every agent follows

1. **One issue per session.** Do not start a second issue until the first has a green pull request.
2. **Read before writing:** this section, §2 (decisions), §3 (layout), §4 (types), then every section the issue's **Spec** line cites.
3. **Interfaces are frozen.** Function names, argument order, struct fields, error codes, event topics and JSON field names come from this guide. If something must change, stop and write the proposed change in the pull request description under "Guide change"; a human approves it.
4. **Never weaken a check to pass a test.** If a test and the guide disagree, the guide wins; if the guide seems wrong, say so in the pull request.
5. **Tests first for contracts.** Write the named tests from §9.2 for the issue, watch them fail, then implement.
6. **Money and time are integers.** `i128` and `u64` in Rust, `bigint` in TypeScript. No floats, no `Number()` on amounts.
7. **Testnet only.** Never put a mainnet passphrase, mainnet RPC URL or real secret anywhere.
8. **No secrets in code or logs.** Read them from env; redact in logs.
9. **Branches** use `type/short-description` (`feat/bond-resolve`, `test/bond-properties`). Commits use conventional titles (`feat(bond): resolve upheld claims`).
10. **No AI attribution** in commits, pull request titles, bodies, comments or code. Write as the team.
11. **Stop and ask** when: the guide is silent, two sections contradict, a dependency issue is not merged, or a command needs network access the environment lacks.

## 17.2 Verification commands by area

| Area | Run before opening the pull request |
|---|---|
| Rust crates and contracts | `cargo fmt --all`, `cargo clippy --workspace --all-targets -- -D warnings`, `cargo test --workspace`, `stellar contract build` |
| TypeScript packages and services | `pnpm -r build`, `pnpm -r lint`, `pnpm test` |
| Anything touching vectors | Both the Rust and TypeScript vector suites |

| Area | Run before opening the pull request |
| --- | --- |
| Deploy scripts | `pnpm deploy:testnet --fresh && pnpm seed:testnet && pnpm smoke:testnet` on a throwaway deployments file |
| Observer, indexer, API | `pnpm dev`, then the relevant `demo:*` script |

## 17.3 Master prompt

Paste this, then the full text of one issue, into any coding agent with repository access:

```
You are implementing one task in the Vaeltrix repository, an open Soroban
protocol on Stellar
where payment providers stake against SLAs and late payments pay clients
automatically.

The implementation guide is docs/implementation-guide.md. It is the source of
truth for every
name, type, error code, event, storage key and API shape. Before writing
code, read:
  - Section 17 (these rules), Section 2 (decisions), Section 3 (layout),
Section 4 (types)
  - Every section listed on the task's "Spec" line

Then:
  1. Restate the task in three bullet points and list the files you will
create or change.
  2. Write the tests named in Section 9.2 or the task's acceptance criteria
first.
  3. Implement until the tests pass. Do not change any public interface
defined in the guide.
  4. Run every verification command in Section 17.2 for the areas you
touched.
  5. Open a pull request on a branch named type/short-description, using the
template in
     Section 17.4. Tick each acceptance criterion you met, and explain any
you did not.

If the guide is silent or contradicts itself, stop and ask with the exact
section numbers.
Never weaken a validation to make a test pass. Never use floats for money or
```

```
time.

TASK:
<paste the full VX-xxx issue here>
```

## 17.4 Pull request template

Save as `.github/pull_request_template.md`:

```
## What
Closes VX-xxx. One paragraph on what changed.

## Guide sections implemented
§x.y, §x.z

## Acceptance criteria
- [ ] (copied from the issue, ticked)

## Tests
Names of new tests and the command that runs them.

## Guide change
None, or the exact proposed edit with section number and reason.

## Testnet
Contract IDs or tx hashes if anything was deployed.
```

## 17.5 Soroban pitfalls agents hit most often

| Pitfall | Do this instead |
|---|---|
| `env.mock_all_auths()` in every test hides missing `require_auth` calls | Use it only in setup; assert auth in dedicated tests with `mock_auths` and check `env.auths()` |
| Calling `require_auth` after reading or writing state | Call it first, before any storage access |
| Treating `ledger().timestamp()` as milliseconds | It is seconds; multiply by 1,000 only when comparing with payment timestamps (D1) |

| Pitfall | Do this instead |
|---|---|
| Enum variants with named fields in `#[contracttype]` | Not supported; wrap a struct in a tuple variant (§4.1) |
| `symbol_short!` with more than 9 characters | Compile error; keep topics short (§8.4) |
| Unbounded `Vec` in storage | Bound every list (signers 5, observers 32, periods 90) |
| Forgetting TTL extension | Extend on every touch (§8.3); test it |
| `panic!` or `unwrap` for business errors | Return `Err(Error::X)`; reserve panics for host failures |
| i128 in JSON for CLI or bindings | Pass as a string |
| Assuming the indexer is current before moving money | Simulate against the contract first (§11.1) |
| Using server time for verdicts | Use ledger close time (§12.6) |
| Re-submitting after a timeout | Persist the tx hash and poll it (§12.10) |
| Hand-editing generated bindings | Regenerate with `pnpm bindings` |

# 18. Deferred items and open questions

Nothing below blocks the weekend demo, but each item must have an owner before the V0 to V1 gate (spec §19.3).

## 18.1 Questions this guide raised

- [ ] Template 1 amount range: spec table says 10 to 10,000 USDC, spec JSON says 1 to 1,000 USDC (7 decimals). Which is intended? (§6.10)
- [ ] Is the corrected exit cooldown (§6.4) acceptable to providers? It adds the 7-day pause ceiling plus target and grace, so about 7.3 days for template 2. A lower MAX_PAUSE_MS would shorten it.
- [ ] Should Path B also cover no-completion, using a provider-signed `ReceiverAccepted` as proof the provider knew about the payment? (§2 D2)
- [ ] Coverage ratio definition (§13.5) is a V0 proposal. Confirm before wallets rely on the score.

- [ ] Pause history keeps only the latest window (§2 D6). Acceptable for V0, or store a bounded list?
- [ ] Should `commit` require `verified` on testnet, or only on mainnet?
- [ ] Delegated claims make the observer the bond payer. Should the protocol fund observer bonds from a fee instead (spec §15.4)?

## 18.2 Open questions carried from the spec (§20.1)

- [ ] Which two anchors pilot V0, on which corridors?
- [ ] Will at least one wallet commit to routing preference?
- [ ] Is 60 minutes realistic for NG, KE and PH bank payouts, or start at 4 hours?
- [ ] Can NIBSS NIP or M-Pesa provide a signed or queryable completion reference?
- [ ] Legal characterisation of penalties per launch jurisdiction.
- [ ] File automatically for every breach, or only on client request?
- [ ] Who runs the first independent observers in V1?
- [ ] Standalone protocol or a performance module alongside Sylox, sharing provider identity?

## 18.3 Deferred work by version

| Item | Version | Where it plugs in |
|---|---|---|
| Multi-observer quorum, bonded observers, `report_observer` slashing | V1 | registry observers, bond `attest`, new slashing function |
| Filing observer excluded from voting on its own claim | V1 | bond `attest` |
| Batch roots matched across a quorum before stats are final | V1 | scoreboard `anchor_batch` |
| Flat and graduated penalties | V1 | new `PenaltyModel` arms (§4.1) |
| Uptime and quote-honouring metrics | V1 | new `Metric` arms and templates |
| Statistical percentile penalties | V2 | scoreboard plus bond period settlement |

| Item | Version | Where it plugs in |
|---|---|---|
| SEP-24 and SEP-6 flows | V2 | new `EventKind` start events |
| Penalty pass-through to end users | V2 | template flag, client obligations |
| Rail-signed completion events | V3 | new evidence mode |
| Permissionless observer set with staking | V3 | registry |
| Formal SEP proposal for the `/info` extension, event format and evidence bundle | after 2 independent anchors | docs |
| Independent security audit | before mainnet | all contracts |

## 18.4 Sources

- Vaeltrix Technical Specification v0.1 (Oct 11, 2026), the attached PDF this guide derives from.
- Stellar CLI releases: CLI 27.x builds optimised wasm by default; constructor arguments and bindings commands.
- Stellar CLI manual: `contract deploy`, `contract asset deploy`, `contract bindings typescript`.
- soroban-sdk 27.0.0 release notes via dependabot: SDK 27 targets protocol 27. Check the official rs-soroban-sdk releases before pinning.